

From Install to Insight: A Hands-On **GPU Lab** for **Java Developers**

Michalis Papadimitriou · Thanos Stratikopoulos · Mary Xekalaki ·
TornadoVM · jitLLM · Hands-on Lab

About us

The TornadoVM team, working on Java for GPUs.



**Michalis
Papadimitriou**

Senior Software Engineer · Neo4j
Research Fellow · University of Manchester
TornadoVM Contributor

PREVIOUSLY

ML Compiler Engineer · OctoAI (acquired by NVIDIA)
Research Software Engineer · Huawei Research Labs

 @mikepapadim

 @mikepapadim

 @mikepapadim.bsky.social

michail.papadimitriou@manchester.ac.uk



**Thanos
Stratikopoulos**

Principal Software Engineer · Beestream Ltd
Research Fellow · University of Manchester
TornadoVM Maintainer

 @stratika

 @thanos_str

 @stratika.bsky.social

thanos.stratikopoulos@beestream.ai



**Mary
Xekalaki**

Principal Software Engineer · Beestream Ltd
Research Associate · University of Manchester
TornadoVM Maintainer

 @mairooni

 @MXekalaki

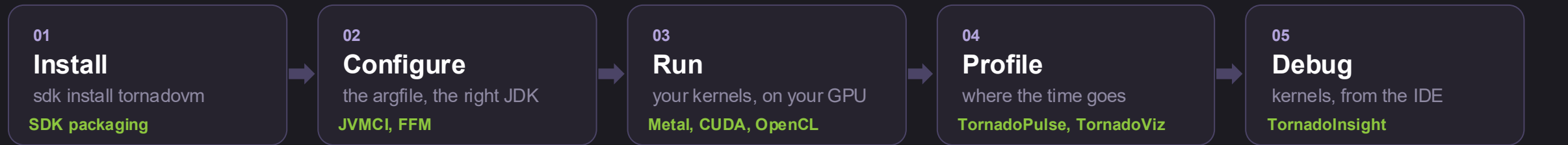
 @maryxek.bsky.social/

mary.xekalaki@beestream.ai

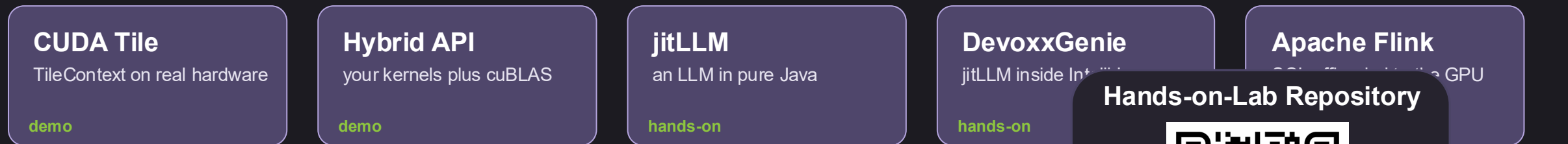
What you'll leave with

Hands-on lab, we have prepared 13 demos ready to run on your own laptop.

Hands-on, on your laptop



Advanced session, on the NVIDIA machine



You leave with:

A working TornadoVM setup, Java kernels you ran, profiled and debugged, and a local LLM in your IDE.
Easter eggs: GPU-accelerated Apache Flink, and TornadoVM for Kotlin.

jitLLM is formerly GPULLama3.java: github.com/beehive-lab/jitllm. The old URLs redirect.

tornadovm.org

Hands-on-Lab Repository



Scan for Lab X

Thirteen releases since the last Devovxx

At Devovx 2025 the latest was 1.1.1. These are the ones today's demos depend on.



Cadence 1.1.1 to 2.0.0 took five months. Since then: 14 releases in 10 months. Each release is one click-away (CI-driven).

One SDK from JDK 21 to JDK 27

JDK 27 removed the JDK-internal JVMCI module, so did we. TornadoVM now ships its own.

01

JDK 21

The platform JVMCI and -XX:+EnableJVMCI, exactly as before.

02

JDK 22 – 27+

No EnableJVMCI: it is a fatal option there. The vendored JVMCI joins as an ordinary module.

What it means for you

One SDK family covers JDK 21 to 27, but the launcher flags differ per JDK, so an argfile is not JDK-neutral. Switch JDK, then run `tornado --generate-argfile`. A stale argfile (Unrecognized VM option EnableJVMCI) is failure #1 in every lab.

From JNI to FFM: what the port bought

PR #1058 replaced hand-written JNI with `java.lang.foreign`. Modern Java stack.

7,400

lines of C / C++ deleted

OpenCL SDK now ships an empty lib/

8

native libraries in the CUDA SDK

only CUTLASS and cuDNN SDPA remain

+3.2%

jitLLM decode

105.09 → 108.45 tok/s, RTX 4090

0

changes to your code

tornado-api is untouched

The CUDA backend, feature by feature

Java → Tornado IR → CUDA C → NVRTC → cubin, for sm_50 to sm_120.

Inside a kernel

Kernel authoring	<code>@Parallel · KernelContext · TileContext</code>
Shared memory, barriers	<code>allocate*LocalArray · localBarrier · globalBarrier</code>
Warp shuffles and votes	<code>simdShuffleDown · simdSum · simdAny · simdBallot</code>
Atomics	<code>atomicAdd · atomicCAS · atomicExchange · Min · Max</code>
Async copy	<code>asyncCopyToLocal · Commit · WaitGroup</code>
Tensor-core MMA	<code>mma · mmaBF16 · mmaInt8 · mmaFP8E4M3 · mmaFP8E5M2</code>
Narrow types	<code>FP8Array · BFloat16Array · HalfFloatArray · Half2</code>

Around the kernels

CUDA graphs	<code>withCUDAGraph()</code>
Multi-stream	<code>withIntraPlanConcurrency()</code>
Pinned H2D staging	<code>withStagedTransfers()</code>
NVIDIA libraries	<code>libraryTask(): cuBLAS · cuBLASLt · cuFFT · cuDNN · cuSPARSE · CUTLASS · cuRAND</code>
NVRTC profiles	<code>default · fast · debug · repro</code>
Profiling	<code>Nsight Systems and Compute · NVTX on library tasks</code>

Not exposed, by design: `cudaMalloc` and `cudaMemcpy`. Memory belongs to the task graph: `transferToDevice`, `transferToHost`, `persistOnDevice`.

Java kernels vs hand-written CUDA

Kernel time only (nsys) vs hand-written CUDA, same launch geometry. RTX 4090, TornadoVM 7.1.0.

98%

of CUDA C++ speed

register-tiled FP32 GEMM (1.02×)

≤3%

memory-bound gap

identical memory-sector counts

1.15×

faster, compute-bound

JIT specialises on argument values

Metal backend: Apple Silicon, same Java

One method, three generated languages. Which one you get depends on the SDK you installed.

Same Java, a third target

@Parallel, KernelContext and TaskGraph code runs unchanged. The JIT emits Metal Shading Language, and --printKernel shows it.

Nothing extra to install

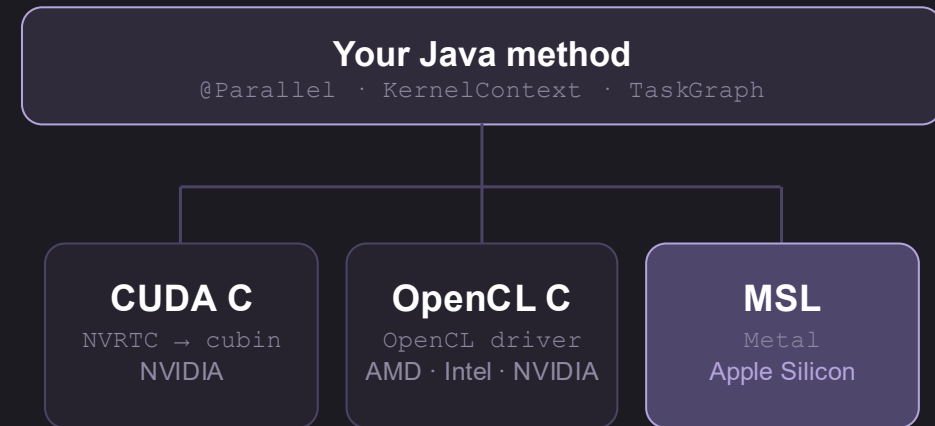
Pick the -metal SDK. The backend uses what macOS already ships.

Even the bridge is Java

Metal is an Objective-C API. TornadoVM reaches it through java.lang.foreign, without a line of Objective-C.

What stays CUDA-only

TileContext and NVIDIA library tasks need the CUDA backend. Plan options such as withCUDAGraph() are no-ops on Metal, so the code stays portable.



```
$ sdk install tornadovm 7.1.0-jdk22plus-metal
```

Added in 3.0.0 / 4.0.0 (Feb–Apr 2026). Apple Silicon only.

TornadoVM

recap

A Java framework that JIT-compiles selected methods to GPU kernels and manages the device around them.

The TornadoVM ecosystem

Any JDK from 21 to 27, three GPU backends, and the tools and integrations around them.

JDK 21-27 DISTRIBUTIONS

OpenJDK	Microsoft JDK
GraalVM	Azul Zulu
Eclipse Temurin	SapMachine
Red Hat Mandrel	

runs on any JDK



compiles per GPU

HARDWARE BACKENDS



OpenCL

AMD · Intel · NVIDIA



CUDA

NVIDIA



Metal

Apple Silicon

DEVELOPER TOOLS

-  TornadoVMPulse
-  TornadoViz
-  TornadoInsight
-  NVIDIA Nsight Systems

DOCKER IMAGES



NVIDIA and Intel images

DISTRIBUTION



Maven Central · SDKMAN!

INTEGRATIONS

 Spring Boot	 Quarkus
 LangChain4j	 Flink

Active since 2018 · 30+ contributors · 30+ research papers · used by the European Space Agency (ESA)

TornadoVM in a nutshell

Runs on the JDK you already have. Not a JVM, not a binding layer.

Your Java application

ordinary bytecode, still running on the JVM

Tornado API

@Parallel · KernelContext · TileContext · TaskGraph · TornadoExecutionPlan

Runtime

task graph · transfer tracking · device memory · CUDA graphs · streams · NVTX

JIT compiler

bytecode → Tornado IR

CUDA backend

SIMT + CUDA Tile

OpenCL

AMD · Intel · NVIDIA

Metal

Apple Silicon

Library providers

cuBLAS · cuFFT · cuDNN · ...

Four ways to program the GPU from Java

From least to most control, plus the vendor's own kernels. Mix them freely in one graph.

01

@Parallel

Annotate a loop. The compiler maps iterations to GPU threads.

easiest start

02

KernelContext

The CUDA thread model in Java: ids, shared memory, barriers, warp shuffles, mma.sync.

full control

03

TileContext

Describe one tile block. The CUDA Tile compiler picks threads and tensor-core instructions.

NEW · CUDA Tile

04

libraryTask

NVIDIA's optimised kernel as a node in the same graph, on the same buffers.

cuBLAS · cuFFT · cuDNN

@Parallel and KernelContext

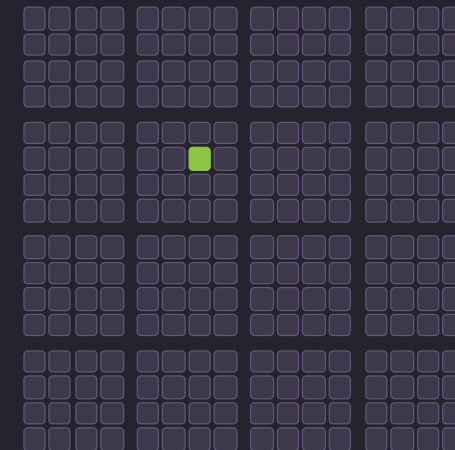
The same $C = A \times B$ two ways, condensed from TileMatrixMultiply.java. A third way comes later.

```

1 // 1. @Parallel: one thread per output element
2 void threadPerElement(HalfFloatArray a, HalfFloatArray b, FloatArray c, int n) {
3     for (@Parallel int row = 0; row < n; row++) {
4         for (@Parallel int column = 0; column < n; column++) {
5             float sum = 0.0f;
6             for (int inner = 0; inner < n; inner++)
7                 sum += a.get(row * n + inner).getFloat32()
8                     * b.get(inner * n + column).getFloat32();
9             c.set(row * n + column, sum);
10        }
11    }
12
13 // grid: new WorkerGrid2D(n, n)
14

```

Output matrix c, drawn as 4 × 4 tiles of 4 × 4 elements



@Parallel

unit of work: one element

Written by

thread indices	TornadoVM
shared memory	not used
barriers	not used
tensor-core instruction	not used
threads per block	TornadoVM

grid: **WorkerGrid2D(n, n)**, one thread per element

- ○ @Parallel: you write the body for one element and TornadoVM launches one thread per element of c.

@Parallel and KernelContext

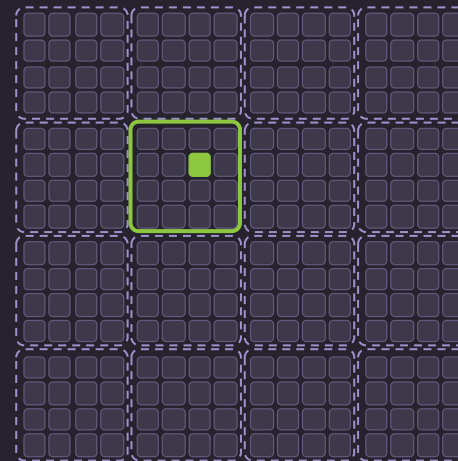
The same $C = A \times B$ two ways, condensed from TileMatrixMultiply.java. A third way comes later.

```

1 // 2. KernelContext: shared-memory tiles, staged by hand
2 void sharedMemoryTiles(KernelContext ctx, HalfFloatArray a, HalfFloatArray b,
3                       FloatArray c, int n) {
4     int lr = ctx.localIdx, lc = ctx.localIdy;
5     int row = ctx.groupIdx * TILE + lr;
6     int col = ctx.groupIdy * TILE + lc;
7     float[] aTile = ctx.allocateFloatLocalArray(TILE * TILE);
8     float[] bTile = ctx.allocateFloatLocalArray(TILE * TILE);
9     float sum = 0.0f;
10    for (int step = 0; step < n / TILE; step++) {
11        aTile[lr*TILE + lc] = a.get(row*n + step*TILE + lc).getFloat32();
12        bTile[lr*TILE + lc] = b.get((step*TILE + lr)*n + col).getFloat32();
13        ctx.localBarrier();
14        for (int i = 0; i < TILE; i++)
15            sum += aTile[lr*TILE + i] * bTile[i*TILE + lc];
16        ctx.localBarrier();
17    }
18    c.set(row * n + col, sum);
19 }

```

Output matrix c, drawn as 4 × 4 tiles of 4 × 4 elements



aTile (local)

bTile (local)

KernelContext

unit of work: one thread, in a 32 × 32 group

Written by

thread indices **you**

shared memory **you**

barriers **you**

tensor-core instruction **not used**

threads per block **you (setLocalWork)**

grid: **WorkerGrid2D(n, n)** + setLocalWork(32, 32, 1)

- ● KernelContext: you compute thread and group indices, stage tiles in local memory, and place both barriers yourself.

CUDA Tile from Java: TileContext

cuTile for Java, through TornadoVM

Program the tile, not the thread.

Your Java kernel gets a `TileContext`. You describe what one tile block computes; TornadoVM lowers each `tc.*` call to cuTile, and NVIDIA's tile compiler maps it to the GPU.

CUDA 13.3+

sm_80+

You write

Tile shapes

$T_M \times T_N \times T_K$

Which tile this block owns

`tc.bidX(), tc.bidY()`

Math on whole tiles

`load · mma · store`

Tile compiler decides

- Threads per block
- Thread indices
- Shared memory & barriers
- Tensor-core instructions

Java + TileContext

TornadoVM JIT

cuTile · ct::*

NVIDIA tile compiler

GPU tensor cores

From threads to tiles

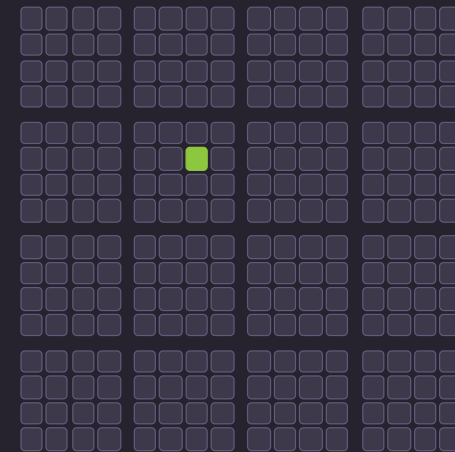
The same $C = A \times B$ written three ways, condensed from TileMatrixMultiply.java.

```

1 // 1. @Parallel: one thread per output element
2 void threadPerElement(HalfFloatArray a, HalfFloatArray b,
3                       FloatArray c, int n) {
4     for (@Parallel int row = 0; row < n; row++)
5         for (@Parallel int column = 0; column < n; column++) {
6             float sum = 0.0f;
7             for (int inner = 0; inner < n; inner++)
8                 sum += a.get(row * n + inner).getFloat32()
9                     * b.get(inner * n + column).getFloat32();
10            c.set(row * n + column, sum);
11        }
12 }
13
14 // grid: new WorkerGrid2D(n, n)

```

Output matrix c, drawn as 4×4 tiles of 4×4 elements



@Parallel

unit of work: one element

Written by

thread indices	TornadoVM
shared memory	not used
barriers	not used
tensor-core instruction	not used
threads per block	TornadoVM

grid: **WorkerGrid2D(n, n)**, one thread per element

- ○ ○ @Parallel: you write the body for one element and TornadoVM launches one thread per element of c.

From threads to tiles

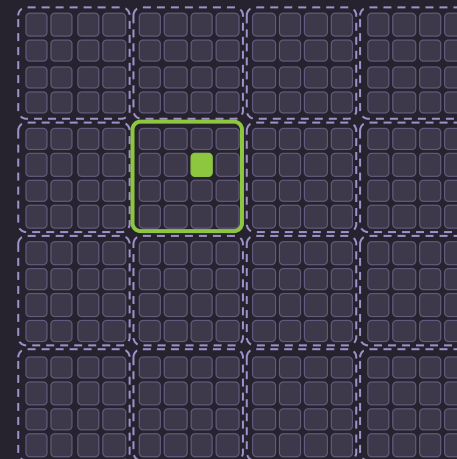
The same $C = A \times B$ written three ways, condensed from TileMatrixMultiply.java.

```

1 // 2. KernelContext: shared-memory tiles, staged by hand
2 void sharedMemoryTiles(KernelContext ctx, HalfFloatArray a,
3                       HalfFloatArray b, FloatArray c, int n) {
4     int lr = ctx.localIdx, lc = ctx.localIdy;
5     int row = ctx.groupIdx * TILE + lr;
6     int col = ctx.groupIdy * TILE + lc;
7     float[] aTile = ctx.allocateFloatLocalArray(TILE * TILE);
8     float[] bTile = ctx.allocateFloatLocalArray(TILE * TILE);
9     float sum = 0.0f;
10    for (int step = 0; step < n / TILE; step++) {
11        aTile[lr*TILE + lc] = a.get(row*n + step*TILE + lc).getFloat32();
12        bTile[lr*TILE + lc] = b.get((step*TILE + lr)*n + col).getFloat32();
13        ctx.localBarrier();
14        for (int i = 0; i < TILE; i++)
15            sum += aTile[lr*TILE + i] * bTile[i*TILE + lc];
16        ctx.localBarrier();
17    }
18    c.set(row * n + col, sum);
19 }

```

Output matrix c, drawn as 4×4 tiles of 4×4 elements



aTile (local)

bTile (local)

KernelContext

unit of work: one thread, in a 32×32 group

Written by

thread indices **you**

shared memory **you**

barriers **you**

tensor-core instruction **not used**

threads per block **you (setLocalWork)**

grid: **WorkerGrid2D(n, n)** + setLocalWork(32, 32, 1)

- ● ○ KernelContext: you compute thread and group indices, stage tiles in local memory, and place both barriers yourself.

From threads to tiles

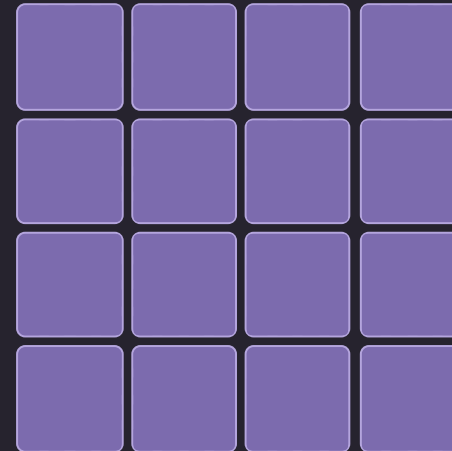
The same $C = A \times B$ written three ways, condensed from TileMatrixMultiply.java.

```

1 // 3. TileContext: a tile is the unit of work
2 void tiles(TileContext tc, HalfFloatArray a, HalfFloatArray b,
3           FloatArray c, int m, int n, int k) {
4     PartitionView aView = tc.partition(tc.view(a, m, k), TILE, TILE);
5     PartitionView bView = tc.partition(tc.view(b, k, n), TILE, TILE);
6     PartitionView cView = tc.partition(tc.view(c, m, n), TILE, TILE);
7     int rowBlock = tc.bidX();
8     int columnBlock = tc.bidY();
9     Tile acc = tc.zeros(DType.F32, TILE, TILE);
10    for (int step = 0; step < k / TILE; step++)
11        acc = tc.mma(aView.load(rowBlock, step),
12                    bView.load(step, columnBlock), acc);
13    cView.store(acc, rowBlock, columnBlock);
14 }
15
16 // grid: new WorkerGrid2D(n / TILE, n / TILE)

```

Output matrix c, drawn as 4×4 tiles of 4×4 elements



TileContext

unit of work: one tile block

Written by

thread indices	none exist
shared memory	tile compiler
barriers	tile compiler
tensor-core instruction	tile compiler
threads per block	tile compiler

grid: **WorkerGrid2D(n / 32, n / 32)** counts tile blocks; local work pinned to $1 \times 1 \times 1$

- ● ● TileContext: you write what one tile block does with whole tiles. The tile compiler decides everything in the right-hand column.

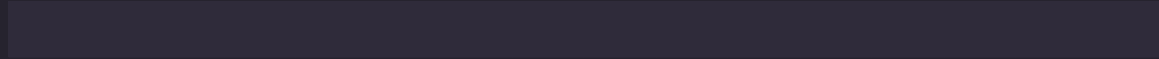
View, partition, load

TileVectorAdd.java: 4,096 floats in tiles of 256. Block indices address tiles, not elements.

```
1 static final int TILE = 256;    // a power-of-two constant
2
3 void vectorAdd(TileContext tc, FloatArray a, FloatArray b,
4               FloatArray c, int n) {
5     PartitionView av = tc.partition(tc.view(a, n), TILE);
6     PartitionView bv = tc.partition(tc.view(b, n), TILE);
7     PartitionView cv = tc.partition(tc.view(c, n), TILE);
8     int block = tc.bidX();
9     cv.store(tc.add(av.load(block), bv.load(block)), block);
10 }
11
12 // host: the grid counts TILE BLOCKS, not threads
13 var worker = new WorkerGrid1D(SIZE / TILE);    // 16
14 var grid = new GridScheduler("tile.add", worker);
15     .task("add", TileVectorAdd::vectorAdd, new TileContext(),
16           a, b, c, SIZE)
```

GPU device memory

a FloatArray(4096)



b FloatArray(4096)



c FloatArray(4096)



- ○ ○ ○ ○ ○ ○ ○ A tile task is one whose first parameter is a TileContext. a, b and c are ordinary TornadoVM arrays in device memory.

View, partition, load

TileVectorAdd.java: 4,096 floats in tiles of 256. Block indices address tiles, not elements.

```
1 static final int TILE = 256;    // a power-of-two constant
2
3 void vectorAdd(TileContext tc, FloatArray a, FloatArray b,
4               FloatArray c, int n) {
5     PartitionView av = tc.partition(tc.view(a, n), TILE);
6     PartitionView bv = tc.partition(tc.view(b, n), TILE);
7     PartitionView cv = tc.partition(tc.view(c, n), TILE);
8     int block = tc.bidX();
9     cv.store(tc.add(av.load(block), bv.load(block)), block);
10 }
11
12 // host: the grid counts TILE BLOCKS, not threads
13 var worker = new WorkerGrid1D(SIZE / TILE);    // 16
14 var grid = new GridScheduler("tile.add", worker);
15     .task("add", TileVectorAdd::vectorAdd, new TileContext(),
16           a, b, c, SIZE)
```

GPU device memory

a FloatArray(4096) view: extent n = 4096, a runtime value

b FloatArray(4096)

c FloatArray(4096)

●●○○○○○ tc.view(a, n) wraps the array as a tensor. Its extent is a runtime value, so the problem size never specialises the kernel.

View, partition, load

TileVectorAdd.java: 4,096 floats in tiles of 256. Block indices address tiles, not elements.

```
1 static final int TILE = 256; // a power-of-two constant
2
3 void vectorAdd(TileContext tc, FloatArray a, FloatArray b,
4               FloatArray c, int n) {
5     PartitionView av = tc.partition(tc.view(a, n), TILE);
6     PartitionView bv = tc.partition(tc.view(b, n), TILE);
7     PartitionView cv = tc.partition(tc.view(c, n), TILE);
8     int block = tc.bidX();
9     cv.store(tc.add(av.load(block), bv.load(block)), block);
10 }
11
12 // host: the grid counts TILE BLOCKS, not threads
13 var worker = new WorkerGrid1D(SIZE / TILE); // 16
14 var grid = new GridScheduler("tile.add", worker);
15     .task("add", TileVectorAdd::vectorAdd, new TileContext(),
16           a, b, c, SIZE)
```

GPU device memory

a FloatArray(4096) view: extent n = 4096, a runtime value

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

b FloatArray(4096) partition: 16 tiles of TILE = 256

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

c FloatArray(4096)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

●●●○○○○○ tc.partition(view, TILE) cuts it into tiles. TILE must be a power-of-two constant: it is part of the kernel's type.

View, partition, load

TileVectorAdd.java: 4,096 floats in tiles of 256. Block indices address tiles, not elements.

```

1 static final int TILE = 256; // a power-of-two constant
2
3 void vectorAdd(TileContext tc, FloatArray a, FloatArray b,
4               FloatArray c, int n) {
5     PartitionView av = tc.partition(tc.view(a, n), TILE);
6     PartitionView bv = tc.partition(tc.view(b, n), TILE);
7     PartitionView cv = tc.partition(tc.view(c, n), TILE);
8     int block = tc.bidX();
9     cv.store(tc.add(av.load(block), bv.load(block)), block);
10 }
11
12 // host: the grid counts TILE BLOCKS, not threads
13 var worker = new WorkerGrid1D(SIZE / TILE); // 16
14 var grid = new GridScheduler("tile.add", worker);
15     .task("add", TileVectorAdd::vectorAdd, new TileContext(),
16           a, b, c, SIZE)

```

GPU device memory

a FloatArray(4096) view: extent n = 4096, a runtime value

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

b FloatArray(4096) partition: 16 tiles of TILE = 256

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

c FloatArray(4096)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

tc.bidX() = 5

●●●●○○○○ The kernel is written for one tile block. tc.bidX() says which one; here, block 5 of 16.

View, partition, load

TileVectorAdd.java: 4,096 floats in tiles of 256. Block indices address tiles, not elements.

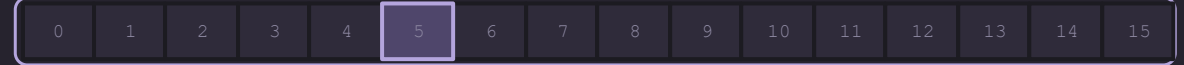
```

1 static final int TILE = 256; // a power-of-two constant
2
3 void vectorAdd(TileContext tc, FloatArray a, FloatArray b,
4               FloatArray c, int n) {
5     PartitionView av = tc.partition(tc.view(a, n), TILE);
6     PartitionView bv = tc.partition(tc.view(b, n), TILE);
7     PartitionView cv = tc.partition(tc.view(c, n), TILE);
8     int block = tc.bidX();
9     cv.store(tc.add(av.load(block), bv.load(block)), block);
10 }
11
12 // host: the grid counts TILE BLOCKS, not threads
13 var worker = new WorkerGrid1D(SIZE / TILE); // 16
14 var grid = new GridScheduler("tile.add", worker);
15     .task("add", TileVectorAdd::vectorAdd, new TileContext(),
16           a, b, c, SIZE)

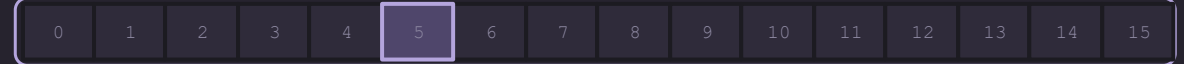
```

GPU device memory

a FloatArray(4096) view: extent n = 4096, a runtime value



b FloatArray(4096) partition: 16 tiles of TILE = 256



Tiles: the values of one tile block (where they live is the compiler's choice)

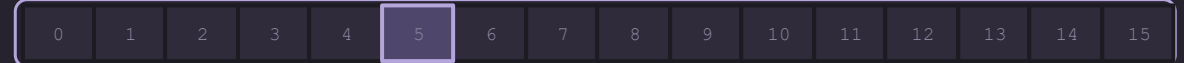
av.load(5)

Tile 256 × F32

bv.load(5)

Tile 256 × F32

c FloatArray(4096)



tc.bidX() = 5

●●●●●○●○● av.load(5) and bv.load(5) bring whole 256-element tiles in. No index arithmetic: the partition view does the addressing.

View, partition, load

TileVectorAdd.java: 4,096 floats in tiles of 256. Block indices address tiles, not elements.

```

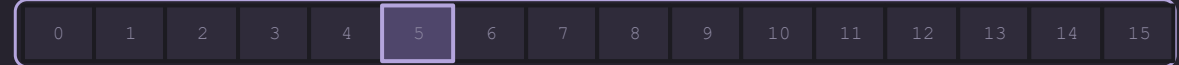
1 static final int TILE = 256; // a power-of-two constant
2
3 void vectorAdd(TileContext tc, FloatArray a, FloatArray b,
4               FloatArray c, int n) {
5     PartitionView av = tc.partition(tc.view(a, n), TILE);
6     PartitionView bv = tc.partition(tc.view(b, n), TILE);
7     PartitionView cv = tc.partition(tc.view(c, n), TILE);
8     int block = tc.bidX();
9     cv.store(tc.add(av.load(block), bv.load(block)), block);
10 }
11
12 // host: the grid counts TILE BLOCKS, not threads
13 var worker = new WorkerGrid1D(SIZE / TILE); // 16
14 var grid = new GridScheduler("tile.add", worker);
15     .task("add", TileVectorAdd::vectorAdd, new TileContext(),
16           a, b, c, SIZE)

```

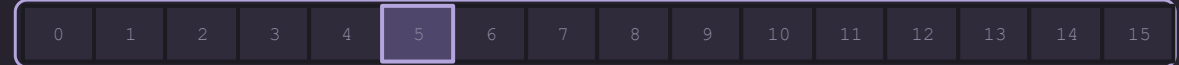
●●●●●●○○ tc.add works on whole tiles and returns a new tile.

GPU device memory

a FloatArray(4096) view: extent n = 4096, a runtime value



b FloatArray(4096) partition: 16 tiles of TILE = 256



Tiles: the values of one tile block (where they live is the compiler's choice)

av.load(5)

Tile 256 × F32

+

bv.load(5)

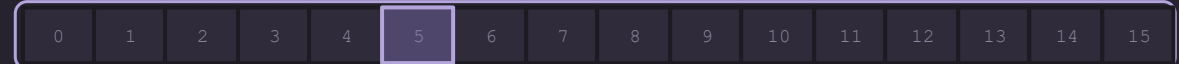
Tile 256 × F32

=

tc.add(...)

Tile 256 × F32

c FloatArray(4096)



tc.bidX() = 5

View, partition, load

TileVectorAdd.java: 4,096 floats in tiles of 256. Block indices address tiles, not elements.

```

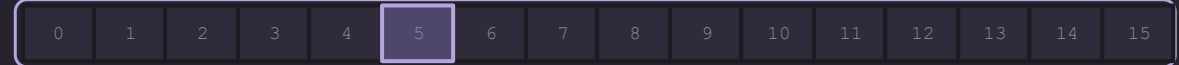
1 static final int TILE = 256; // a power-of-two constant
2
3 void vectorAdd(TileContext tc, FloatArray a, FloatArray b,
4               FloatArray c, int n) {
5     PartitionView av = tc.partition(tc.view(a, n), TILE);
6     PartitionView bv = tc.partition(tc.view(b, n), TILE);
7     PartitionView cv = tc.partition(tc.view(c, n), TILE);
8     int block = tc.bidX();
9     cv.store(tc.add(av.load(block), bv.load(block)), block);
10 }
11
12 // host: the grid counts TILE BLOCKS, not threads
13 var worker = new WorkerGrid1D(SIZE / TILE); // 16
14 var grid = new GridScheduler("tile.add", worker);
15     .task("add", TileVectorAdd::vectorAdd, new TileContext(),
16           a, b, c, SIZE)

```

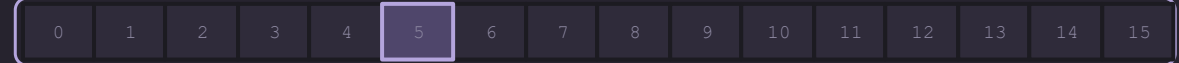
●●●●●●●○ cv.store(tile, 5) writes the result tile back to block 5 of c.

GPU device memory

a FloatArray(4096) view: extent n = 4096, a runtime value



b FloatArray(4096) partition: 16 tiles of TILE = 256



Tiles: the values of one tile block (where they live is the compiler's choice)

av.load(5)

Tile 256 × F32

+

bv.load(5)

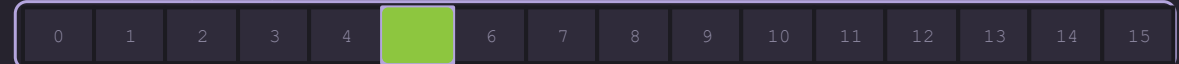
Tile 256 × F32

=

tc.add(...)

Tile 256 × F32

c FloatArray(4096)



tc.bidX() = 5

View, partition, load

TileVectorAdd.java: 4,096 floats in tiles of 256. Block indices address tiles, not elements.

```

1 static final int TILE = 256; // a power-of-two constant
2
3 void vectorAdd(TileContext tc, FloatArray a, FloatArray b,
4               FloatArray c, int n) {
5     PartitionView av = tc.partition(tc.view(a, n), TILE);
6     PartitionView bv = tc.partition(tc.view(b, n), TILE);
7     PartitionView cv = tc.partition(tc.view(c, n), TILE);
8     int block = tc.bidX();
9     cv.store(tc.add(av.load(block), bv.load(block)), block);
10 }
11
12 // host: the grid counts TILE BLOCKS, not threads
13 var worker = new WorkerGrid1D(SIZE / TILE); // 16
14 var grid = new GridScheduler("tile.add", worker);
15     .task("add", TileVectorAdd::vectorAdd, new TileContext(),
16           a, b, c, SIZE)

```

GPU device memory

a FloatArray(4096) view: extent n = 4096, a runtime value

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

b FloatArray(4096) partition: 16 tiles of TILE = 256

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

Tiles: the values of one tile block (where they live is the compiler's choice)

av.load(5)

Tile 256 × F32

+

bv.load(5)

Tile 256 × F32

=

tc.add(...)

Tile 256 × F32

c FloatArray(4096)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

WorkerGrid1D(16): 16 tile blocks, each running the same kernel

- The worker grid counts tile blocks: $4096 / 256 = 16$. No setLocalWork; CUDA Tile owns the thread mapping.

A tensor-core GEMM with tc.mma

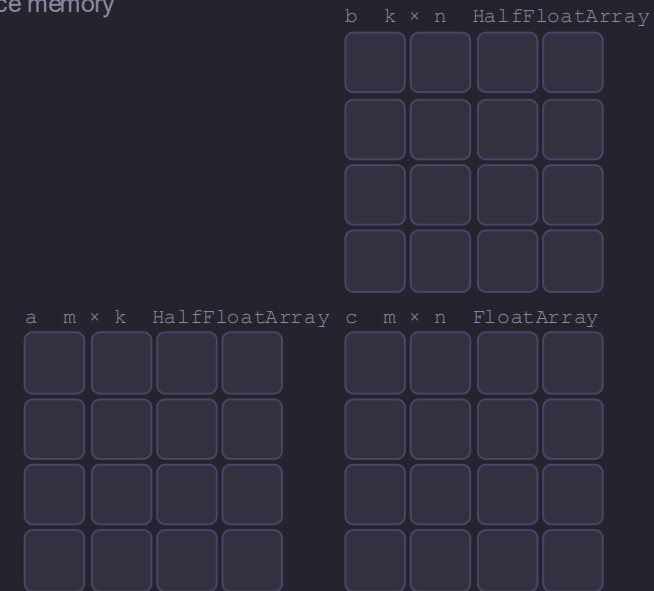
FP16 inputs, FP32 accumulator. Drawn at $n = 128$: each matrix is 4×4 tiles of 32×32 .

```

1 static final int TILE = 32;
2
3 void tiles(TileContext tc, HalfFloatArray a, HalfFloatArray b,
4           FloatArray c, int m, int n, int k) {
5     PartitionView aView = tc.partition(tc.view(a, m, k), TILE, TILE);
6     PartitionView bView = tc.partition(tc.view(b, k, n), TILE, TILE);
7     PartitionView cView = tc.partition(tc.view(c, m, n), TILE, TILE);
8     int rowBlock = tc.bidX();
9     int columnBlock = tc.bidY();
10    Tile acc = tc.zeros(DType.F32, TILE, TILE);
11    for (int step = 0; step < k / TILE; step++)
12        acc = tc.mma(aView.load(rowBlock, step),
13                    bView.load(step, columnBlock), acc);
14    cView.store(acc, rowBlock, columnBlock);
15 }
16
17 // grid: new WorkerGrid2D(n / TILE, n / TILE)

```

GPU device memory



- ○ ○ ○ ○ ○ ○ ○ Three views, each partitioned into 32×32 tiles. The extents m , n and k are runtime values; $TILE$ is a constant.

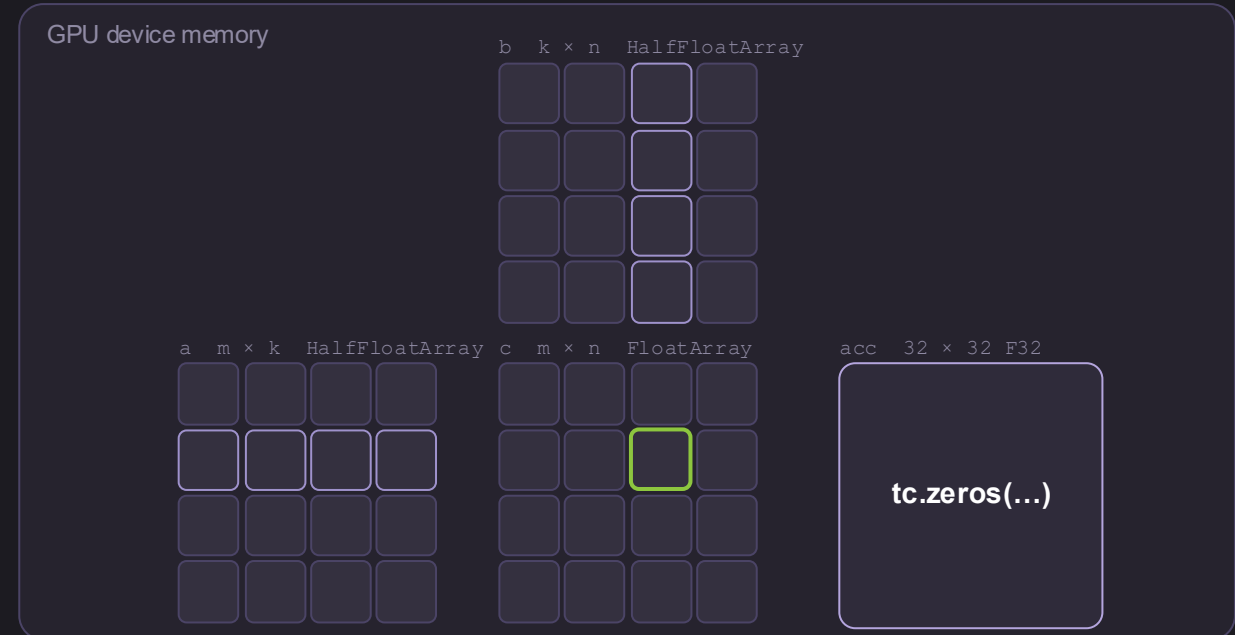
A tensor-core GEMM with tc.mma

FP16 inputs, FP32 accumulator. Drawn at $n = 128$: each matrix is 4×4 tiles of 32×32 .

```

1 static final int TILE = 32;
2
3 void tiles(TileContext tc, HalfFloatArray a, HalfFloatArray b,
4           FloatArray c, int m, int n, int k) {
5     PartitionView aView = tc.partition(tc.view(a, m, k), TILE, TILE);
6     PartitionView bView = tc.partition(tc.view(b, k, n), TILE, TILE);
7     PartitionView cView = tc.partition(tc.view(c, m, n), TILE, TILE);
8     int rowBlock = tc.bidX();
9     int columnBlock = tc.bidY();
10    Tile acc = tc.zeros(DType.F32, TILE, TILE);
11    for (int step = 0; step < k / TILE; step++)
12        acc = tc.mma(aView.load(rowBlock, step),
13                    bView.load(step, columnBlock), acc);
14    cView.store(acc, rowBlock, columnBlock);
15 }
16
17 // grid: new WorkerGrid2D(n / TILE, n / TILE)

```



$tc.bidX() = 1, tc.bidY() = 2$

- This tile block owns one tile of c: (bidX, bidY) = (1, 2). Its accumulator starts as a zero FP32 tile.

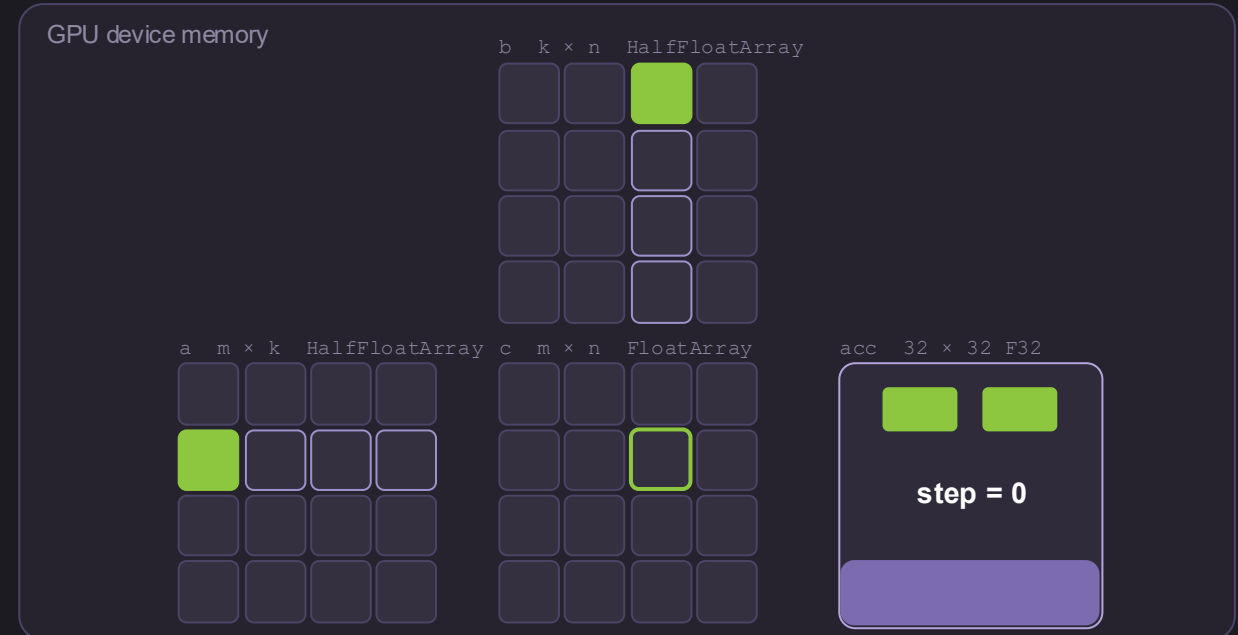
A tensor-core GEMM with tc.mma

FP16 inputs, FP32 accumulator. Drawn at $n = 128$: each matrix is 4×4 tiles of 32×32 .

```

1 static final int TILE = 32;
2
3 void tiles(TileContext tc, HalfFloatArray a, HalfFloatArray b,
4           FloatArray c, int m, int n, int k) {
5     PartitionView aView = tc.partition(tc.view(a, m, k), TILE, TILE);
6     PartitionView bView = tc.partition(tc.view(b, k, n), TILE, TILE);
7     PartitionView cView = tc.partition(tc.view(c, m, n), TILE, TILE);
8     int rowBlock = tc.bidX();
9     int columnBlock = tc.bidY();
10    Tile acc = tc.zeros(DType.F32, TILE, TILE);
11    for (int step = 0; step < k / TILE; step++)
12        acc = tc.mma(aView.load(rowBlock, step),
13                    bView.load(step, columnBlock), acc);
14    cView.store(acc, rowBlock, columnBlock);
15 }
16
17 // grid: new WorkerGrid2D(n / TILE, n / TILE)

```



$tc.bidX() = 1$, $tc.bidY() = 2$

●●●○○○○○ Step 0 loads $a(1, 0)$ and $b(0, 2)$. $tc.mma(a, b, acc)$ returns $acc + a \times b$ as a new tile: FP16 operands, FP32 sum.

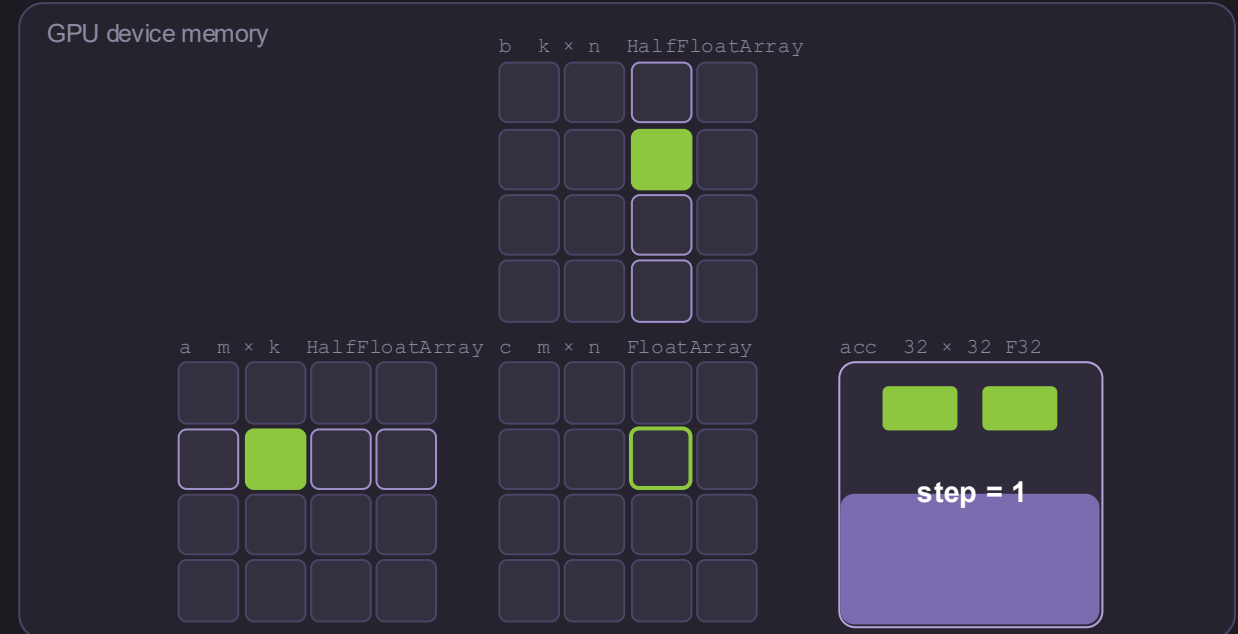
A tensor-core GEMM with tc.mma

FP16 inputs, FP32 accumulator. Drawn at $n = 128$: each matrix is 4×4 tiles of 32×32 .

```

1 static final int TILE = 32;
2
3 void tiles(TileContext tc, HalfFloatArray a, HalfFloatArray b,
4           FloatArray c, int m, int n, int k) {
5     PartitionView aView = tc.partition(tc.view(a, m, k), TILE, TILE);
6     PartitionView bView = tc.partition(tc.view(b, k, n), TILE, TILE);
7     PartitionView cView = tc.partition(tc.view(c, m, n), TILE, TILE);
8     int rowBlock = tc.bidX();
9     int columnBlock = tc.bidY();
10    Tile acc = tc.zeros(DType.F32, TILE, TILE);
11    for (int step = 0; step < k / TILE; step++)
12        acc = tc.mma(aView.load(rowBlock, step),
13                    bView.load(step, columnBlock), acc);
14    cView.store(acc, rowBlock, columnBlock);
15 }
16
17 // grid: new WorkerGrid2D(n / TILE, n / TILE)

```



$tc.bidX() = 1, tc.bidY() = 2$

●●●●○○○○ step = 1: the loop walks along k, one more product into the accumulator. It lowers to `ct::irange`.

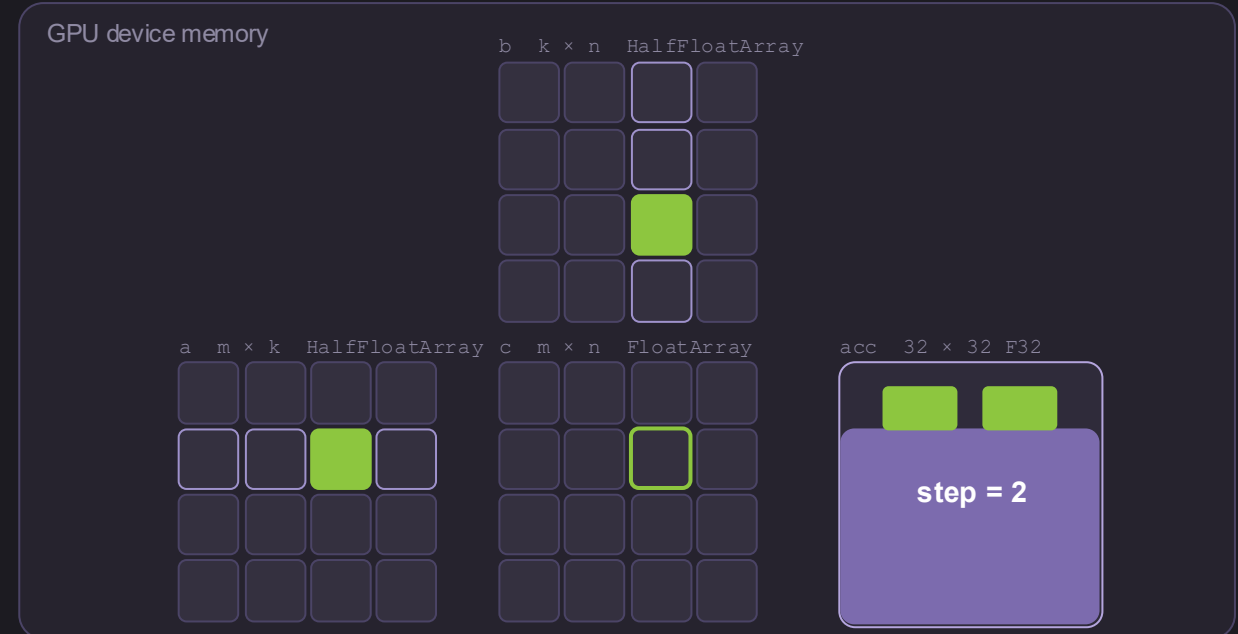
A tensor-core GEMM with tc.mma

FP16 inputs, FP32 accumulator. Drawn at $n = 128$: each matrix is 4×4 tiles of 32×32 .

```

1 static final int TILE = 32;
2
3 void tiles(TileContext tc, HalfFloatArray a, HalfFloatArray b,
4           FloatArray c, int m, int n, int k) {
5     PartitionView aView = tc.partition(tc.view(a, m, k), TILE, TILE);
6     PartitionView bView = tc.partition(tc.view(b, k, n), TILE, TILE);
7     PartitionView cView = tc.partition(tc.view(c, m, n), TILE, TILE);
8     int rowBlock = tc.bidX();
9     int columnBlock = tc.bidY();
10    Tile acc = tc.zeros(DType.F32, TILE, TILE);
11    for (int step = 0; step < k / TILE; step++)
12        acc = tc.mma(aView.load(rowBlock, step),
13                    bView.load(step, columnBlock), acc);
14    cView.store(acc, rowBlock, columnBlock);
15 }
16
17 // grid: new WorkerGrid2D(n / TILE, n / TILE)

```



$tc.bidX() = 1, tc.bidY() = 2$

●●●●●○ step = 2: the loop walks along k, one more product into the accumulator. It lowers to `ct::irange`.

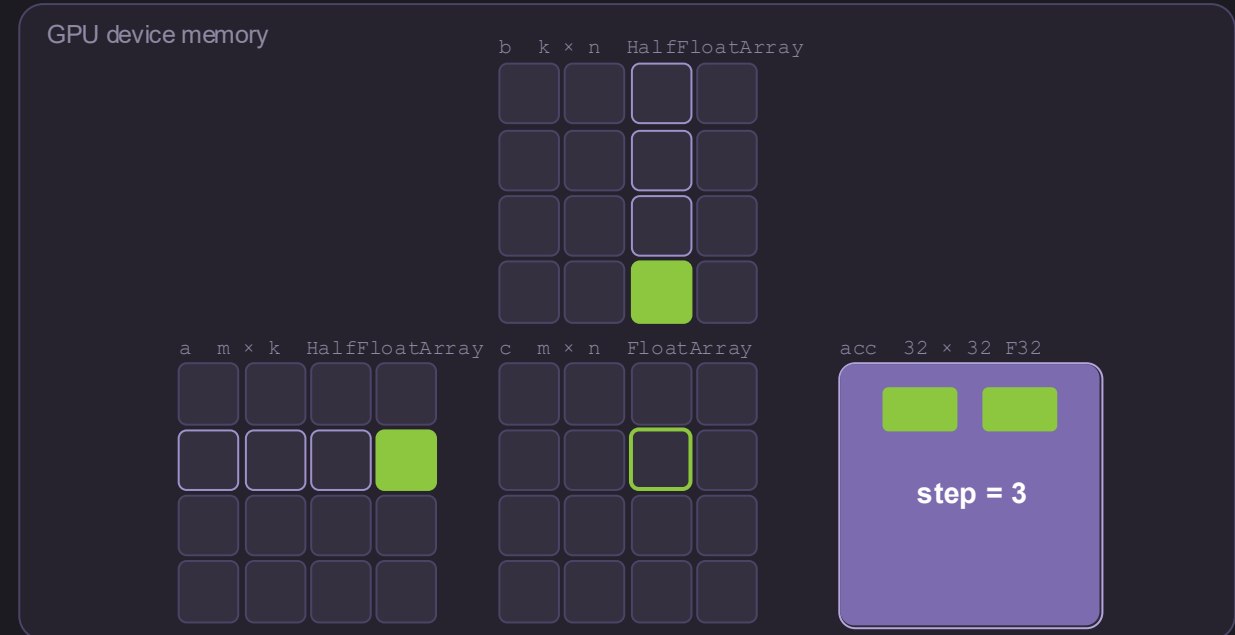
A tensor-core GEMM with tc.mma

FP16 inputs, FP32 accumulator. Drawn at $n = 128$: each matrix is 4×4 tiles of 32×32 .

```

1 static final int TILE = 32;
2
3 void tiles(TileContext tc, HalfFloatArray a, HalfFloatArray b,
4           FloatArray c, int m, int n, int k) {
5     PartitionView aView = tc.partition(tc.view(a, m, k), TILE, TILE);
6     PartitionView bView = tc.partition(tc.view(b, k, n), TILE, TILE);
7     PartitionView cView = tc.partition(tc.view(c, m, n), TILE, TILE);
8     int rowBlock = tc.bidX();
9     int columnBlock = tc.bidY();
10    Tile acc = tc.zeros(DType.F32, TILE, TILE);
11    for (int step = 0; step < k / TILE; step++)
12        acc = tc.mma(aView.load(rowBlock, step),
13                    bView.load(step, columnBlock), acc);
14    cView.store(acc, rowBlock, columnBlock);
15 }
16
17 // grid: new WorkerGrid2D(n / TILE, n / TILE)

```



$tc.bidX() = 1$, $tc.bidY() = 2$

●●●●●●○○ step = 3: the loop walks along k , one more product into the accumulator. It lowers to `ct::irange`.

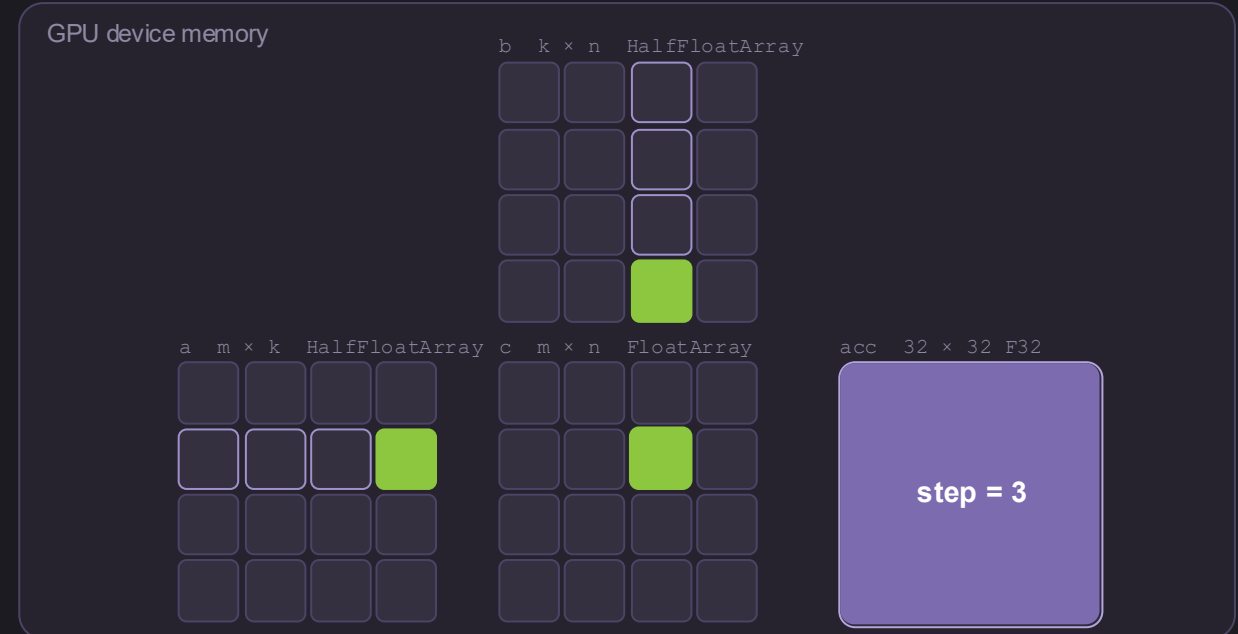
A tensor-core GEMM with tc.mma

FP16 inputs, FP32 accumulator. Drawn at $n = 128$: each matrix is 4×4 tiles of 32×32 .

```

1 static final int TILE = 32;
2
3 void tiles(TileContext tc, HalfFloatArray a, HalfFloatArray b,
4           FloatArray c, int m, int n, int k) {
5     PartitionView aView = tc.partition(tc.view(a, m, k), TILE, TILE);
6     PartitionView bView = tc.partition(tc.view(b, k, n), TILE, TILE);
7     PartitionView cView = tc.partition(tc.view(c, m, n), TILE, TILE);
8     int rowBlock = tc.bidX();
9     int columnBlock = tc.bidY();
10    Tile acc = tc.zeros(DType.F32, TILE, TILE);
11    for (int step = 0; step < k / TILE; step++)
12        acc = tc.mma(aView.load(rowBlock, step),
13                    bView.load(step, columnBlock), acc);
14    cView.store(acc, rowBlock, columnBlock);
15 }
16
17 // grid: new WorkerGrid2D(n / TILE, n / TILE)

```



$tc.bidX() = 1$, $tc.bidY() = 2$

●●●●●●●○ $cView.store(acc, 1, 2)$ writes the finished tile. No thread index, shared memory, barrier or fragment shape anywhere.

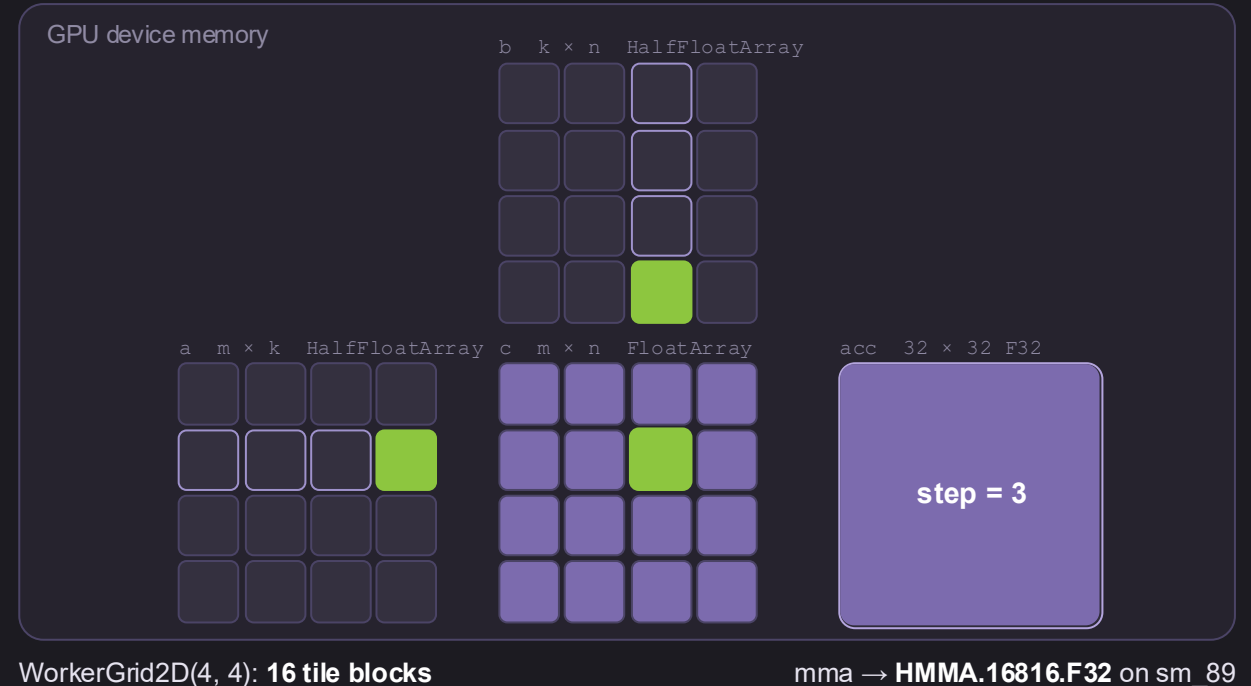
A tensor-core GEMM with tc.mma

FP16 inputs, FP32 accumulator. Drawn at $n = 128$: each matrix is 4×4 tiles of 32×32 .

```

1 static final int TILE = 32;
2
3 void tiles(TileContext tc, HalfFloatArray a, HalfFloatArray b,
4           FloatArray c, int m, int n, int k) {
5     PartitionView aView = tc.partition(tc.view(a, m, k), TILE, TILE);
6     PartitionView bView = tc.partition(tc.view(b, k, n), TILE, TILE);
7     PartitionView cView = tc.partition(tc.view(c, m, n), TILE, TILE);
8     int rowBlock = tc.bidX();
9     int columnBlock = tc.bidY();
10    Tile acc = tc.zeros(DType.F32, TILE, TILE);
11    for (int step = 0; step < k / TILE; step++)
12        acc = tc.mma(aView.load(rowBlock, step),
13                    bView.load(step, columnBlock), acc);
14    cView.store(acc, rowBlock, columnBlock);
15 }
16
17 // grid: new WorkerGrid2D(n / TILE, n / TILE)

```

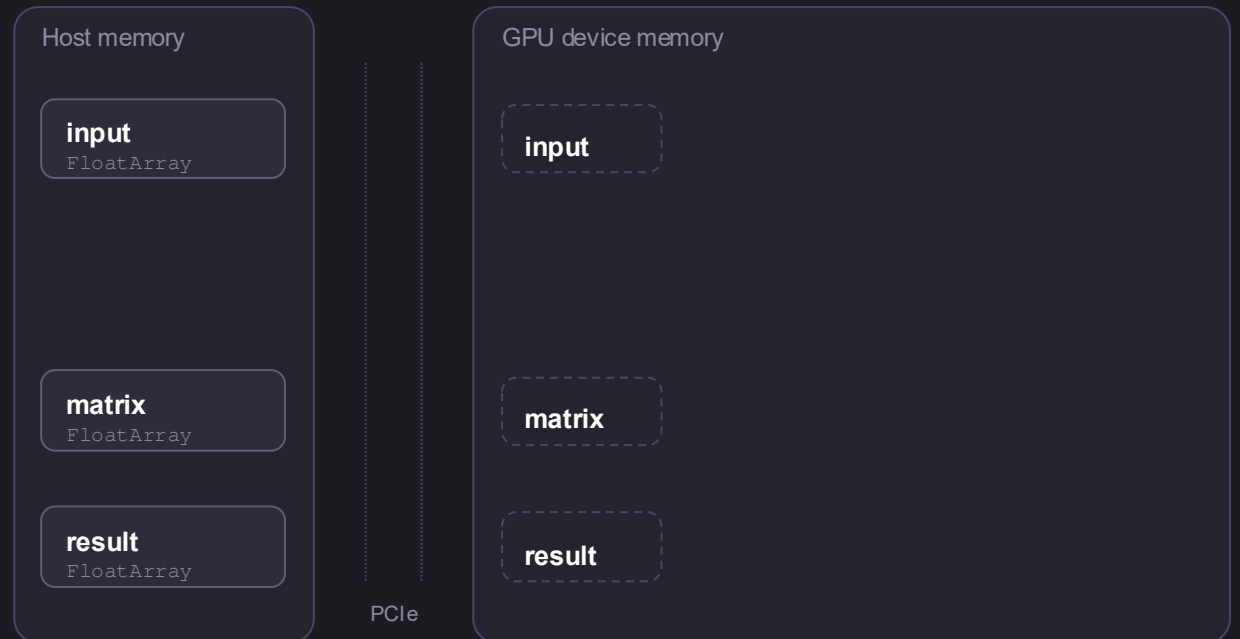


●●●●●●●● The grid has one tile block per tile of c. On Ada (sm_89) the tile compiler picked HMMA.16816.F32 for this mma.

Java, CUDA Tile and cuBLAS in one graph

The chain from the TileContext docs: scale $\rightarrow$ tile $\rightarrow$ gemv $\rightarrow$ bias, captured as one CUDA graph.

```
1 var tg = new TaskGraph("chain")
2   .transferToDevice(EVERY_EXECUTION, input, matrix)
3   .task("scale", Chain::scale, input, scaled, 3.0f)
4   .task("tile", Chain::tileDouble, new TileContext(),
5         scaled, doubled, N)
6   .libraryTask("gemv", CuBlas::cublasSgemv,
7               CUBLAS_OP_T.operation(), m, n, alpha,
8               matrix, lda, doubled, 1, beta, projected, 1)
9   .task("bias", Chain::bias, projected, result, 0.25f)
10  .transferToHost(EVERY_EXECUTION, result);
11
12 try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
13   plan.withGridScheduler(grid).withCUDAGraph().execute();
14 }
```

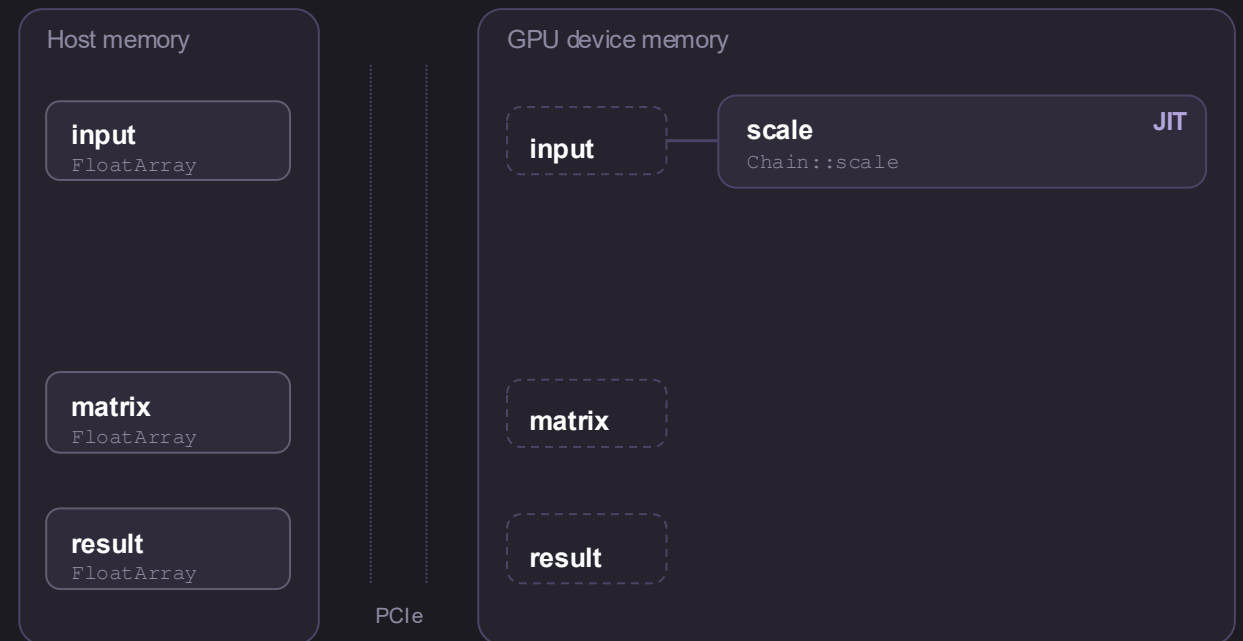


- ○ ○ ○ ○ ○ ○ ○ A TaskGraph starts by naming the host arrays it needs on the GPU. Their device buffers are allocated on the first execute().

Java, CUDA Tile and cuBLAS in one graph

The chain from the TileContext docs: scale → tile → gemv → bias, captured as one CUDA graph.

```
1 var tg = new TaskGraph("chain")
2   .transferToDevice(EVERY_EXECUTION, input, matrix)
3   .task("scale", Chain::scale, input, scaled, 3.0f)
4   .task("tile", Chain::tileDouble, new TileContext(),
5         scaled, doubled, N)
6   .libraryTask("gemv", CuBlas::cublasSgemv,
7               CUBLAS_OP_T.operation(), m, n, alpha,
8               matrix, lda, doubled, 1, beta, projected, 1)
9   .task("bias", Chain::bias, projected, result, 0.25f)
10  .transferToHost(EVERY_EXECUTION, result);
11
12 try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
13   plan.withGridScheduler(grid).withCUDAGraph().execute();
14 }
```



- ● ○ ○ ○ ○ ○ ○ Task 1 is a Java kernel, JIT-compiled to CUDA C. Its output, scaled, is a device buffer the next task reads directly.

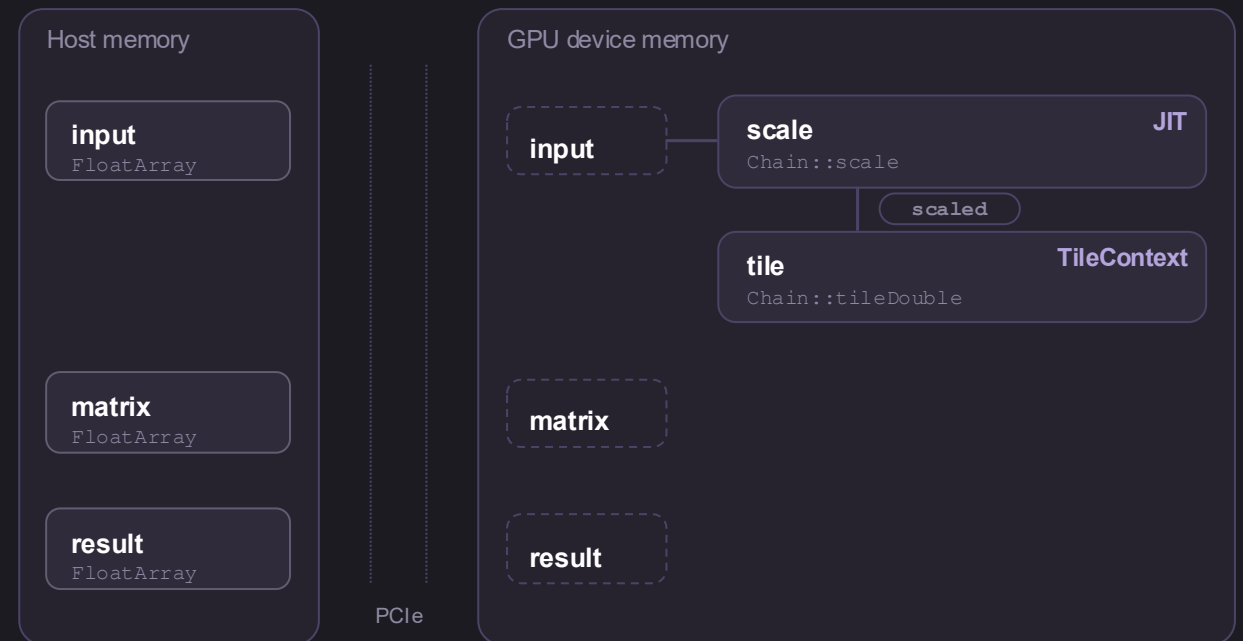
Java, CUDA Tile and cuBLAS in one graph

The chain from the TileContext docs: scale → tile → gemv → bias, captured as one CUDA graph.

```

1 var tg = new TaskGraph("chain")
2   .transferToDevice(EVERY_EXECUTION, input, matrix)
3   .task("scale", Chain::scale, input, scaled, 3.0f)
4   .task("tile", Chain::tileDouble, new TileContext(),
5         scaled, doubled, N)
6   .libraryTask("gemv", CuBlas::cublasSgemv,
7               CUBLAS_OP_T.operation(), m, n, alpha,
8               matrix, lda, doubled, 1, beta, projected, 1)
9   .task("bias", Chain::bias, projected, result, 0.25f)
10  .transferToHost(EVERY_EXECUTION, result);
11
12 try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
13   plan.withGridScheduler(grid).withCUDAGraph().execute();
14 }

```

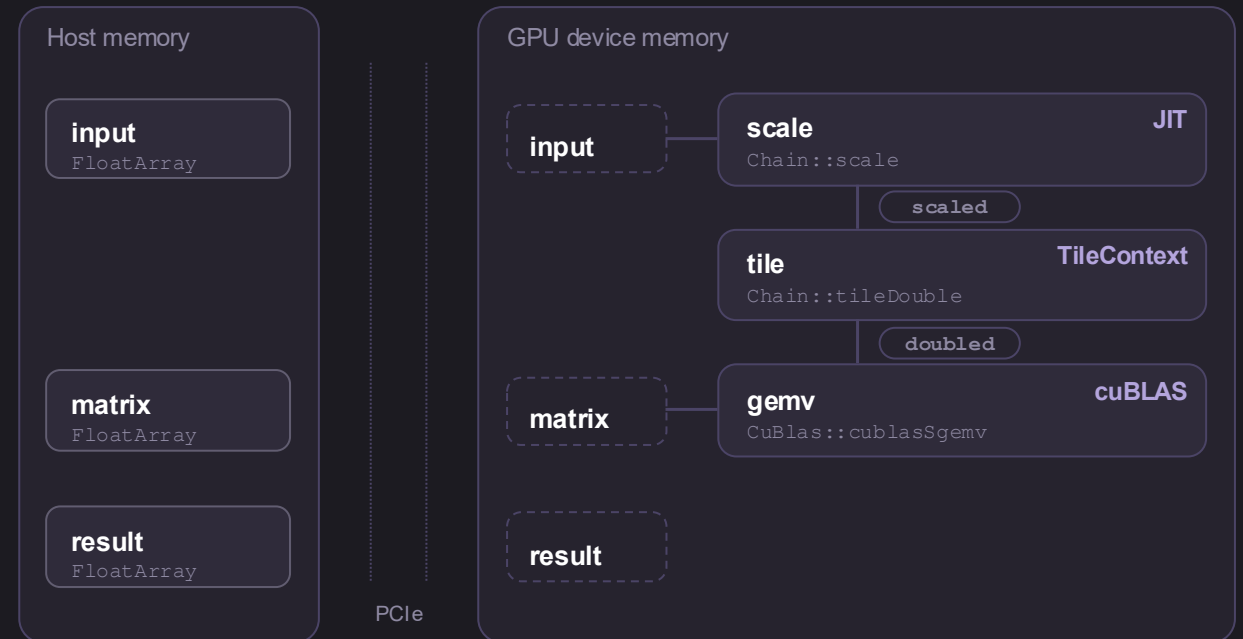


- ● ● ○ ○ ○ ○ ○ Task 2 is a CUDA Tile kernel: its first parameter is a TileContext. It reads scaled from device memory and writes doubled.

Java, CUDA Tile and cuBLAS in one graph

The chain from the TileContext docs: scale → tile → gemv → bias, captured as one CUDA graph.

```
1 var tg = new TaskGraph("chain")
2   .transferToDevice(EVERY_EXECUTION, input, matrix)
3   .task("scale", Chain::scale, input, scaled, 3.0f)
4   .task("tile", Chain::tileDouble, new TileContext(),
5         scaled, doubled, N)
6   .libraryTask("gemv", CuBlas::cublasSgemv,
7               CUBLAS_OP_T.operation(), m, n, alpha,
8               matrix, lda, doubled, 1, beta, projected, 1)
9   .task("bias", Chain::bias, projected, result, 0.25f)
10  .transferToHost(EVERY_EXECUTION, result);
11
12 try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
13   plan.withGridScheduler(grid).withCUDAGraph().execute();
14 }
```

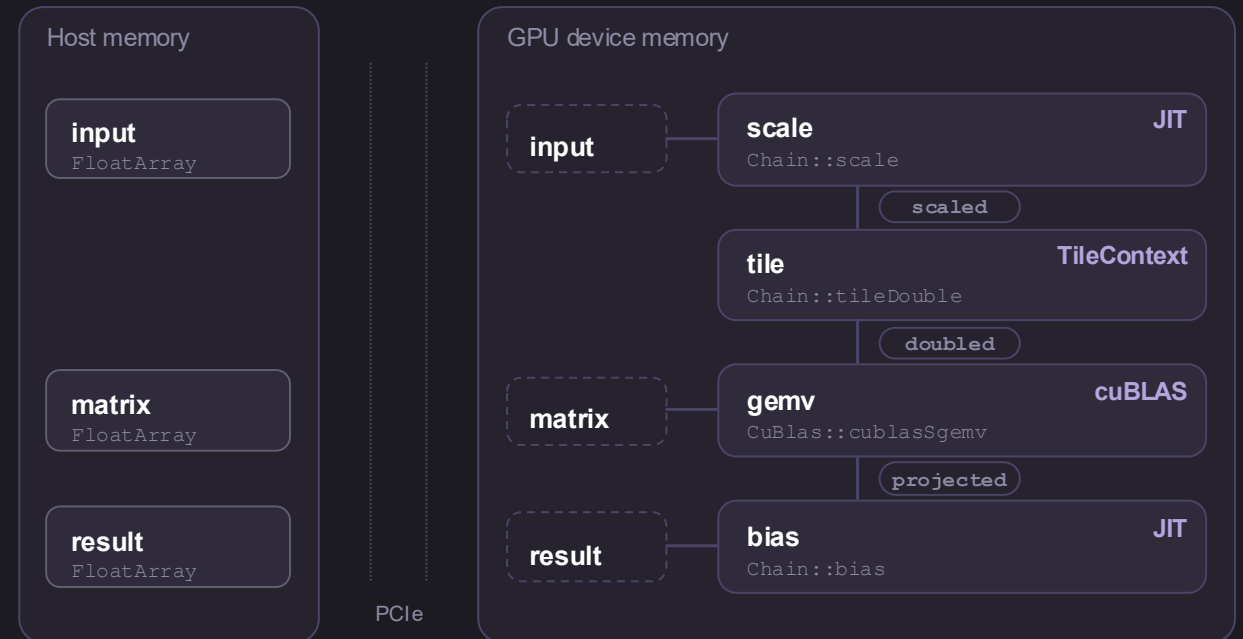


●●●●○○○ Task 3 is cuBLAS, added with libraryTask. It multiplies matrix by doubled from the tile kernel: same buffer, no copy.

Java, CUDA Tile and cuBLAS in one graph

The chain from the TileContext docs: `scale` → `tile` → `gemv` → `bias`, captured as one CUDA graph.

```
1 var tg = new TaskGraph("chain")
2   .transferToDevice(EVERY_EXECUTION, input, matrix)
3   .task("scale", Chain::scale, input, scaled, 3.0f)
4   .task("tile", Chain::tileDouble, new TileContext(),
5         scaled, doubled, N)
6   .libraryTask("gemv", CuBlas::cublasSgemv,
7               CUBLAS_OP_T.operation(), m, n, alpha,
8               matrix, lda, doubled, 1, beta, projected, 1)
9   .task("bias", Chain::bias, projected, result, 0.25f)
10  .transferToHost(EVERY_EXECUTION, result);
11
12 try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
13   plan.withGridScheduler(grid).withCUDAGraph().execute();
14 }
```

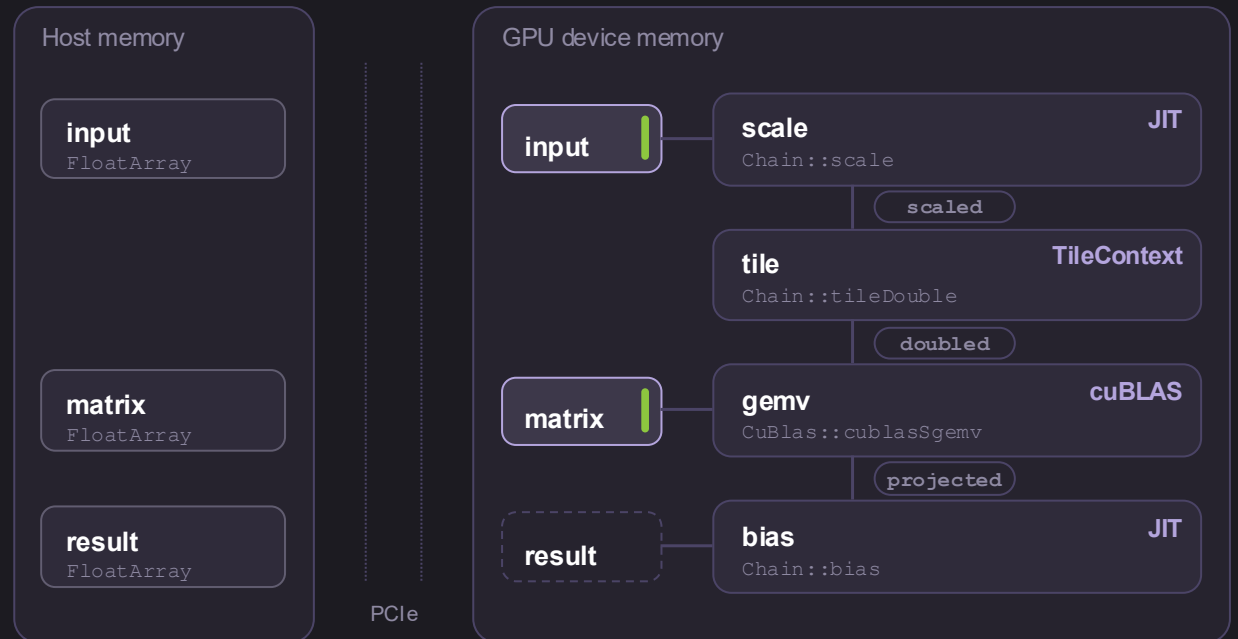


- Task 4 is another Java kernel. The tile task, like a `KernelContext` task, takes its launch shape from the `GridScheduler`.

Java, CUDA Tile and cuBLAS in one graph

The chain from the TileContext docs: scale → tile → gemv → bias, captured as one CUDA graph.

```
1 var tg = new TaskGraph("chain")
2   .transferToDevice(EVERY_EXECUTION, input, matrix)
3   .task("scale", Chain::scale, input, scaled, 3.0f)
4   .task("tile", Chain::tileDouble, new TileContext(),
5         scaled, doubled, N)
6   .libraryTask("gemv", CuBlas::cublasSgemv,
7               CUBLAS_OP_T.operation(), m, n, alpha,
8               matrix, lda, doubled, 1, beta, projected, 1)
9   .task("bias", Chain::bias, projected, result, 0.25f)
10  .transferToHost(EVERY_EXECUTION, result);
11
12 try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
13   plan.withGridScheduler(grid).withCUDAGraph().execute();
14 }
```

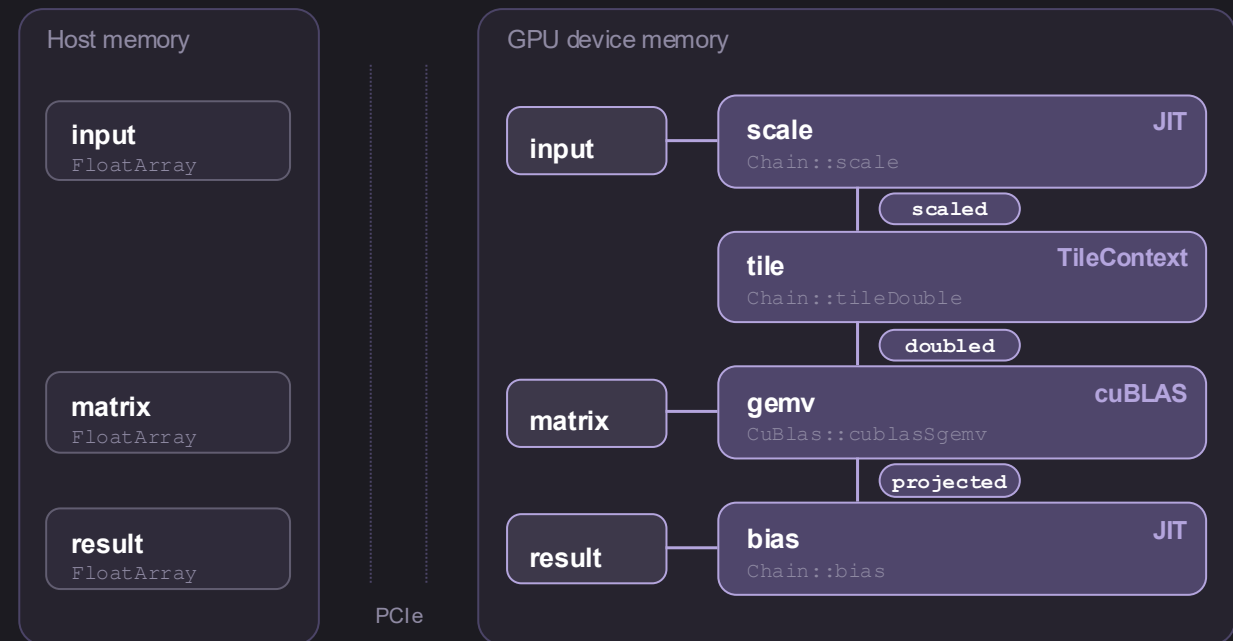


●●●●●●○○ execute() starts by copying input and matrix into their device buffers.

Java, CUDA Tile and cuBLAS in one graph

The chain from the TileContext docs: scale → tile → gemv → bias, captured as one CUDA graph.

```
1 var tg = new TaskGraph("chain")
2   .transferToDevice(EVERY_EXECUTION, input, matrix)
3   .task("scale", Chain::scale, input, scaled, 3.0f)
4   .task("tile", Chain::tileDouble, new TileContext(),
5         scaled, doubled, N)
6   .libraryTask("gemv", CuBlas::cublasSgemv,
7               CUBLAS_OP_T.operation(), m, n, alpha,
8               matrix, lda, doubled, 1, beta, projected, 1)
9   .task("bias", Chain::bias, projected, result, 0.25f)
10  .transferToHost(EVERY_EXECUTION, result);
11
12 try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
13   plan.withGridScheduler(grid).withCUDAGraph().execute();
14 }
```

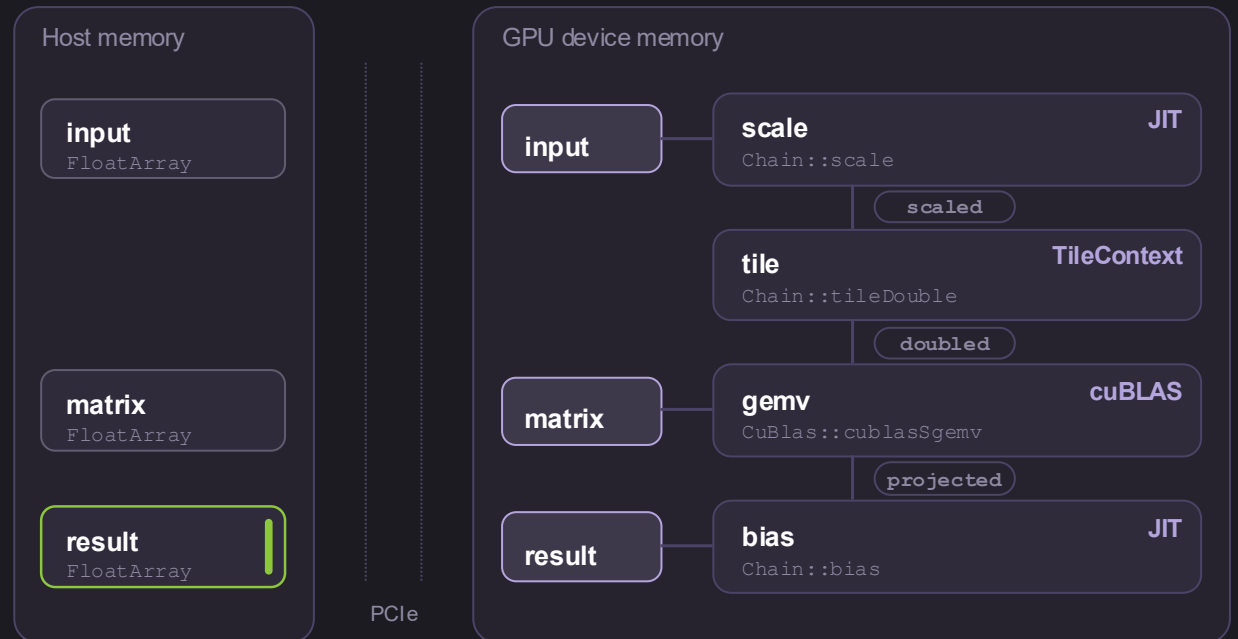


●●●●●●●○ Then the four tasks run in order on one stream. withCUDAGraph() captures all of them, tile and cuBLAS included, as one CUDA graph.

Java, CUDA Tile and cuBLAS in one graph

The chain from the TileContext docs: `scale` → `tile` → `gemv` → `bias`, captured as one CUDA graph.

```
1 var tg = new TaskGraph("chain")
2   .transferToDevice(EVERY_EXECUTION, input, matrix)
3   .task("scale", Chain::scale, input, scaled, 3.0f)
4   .task("tile", Chain::tileDouble, new TileContext(),
5       scaled, doubled, N)
6   .libraryTask("gemv", CuBlas::cublasSgemv,
7       CUBLAS_OP_T.operation(), m, n, alpha,
8       matrix, lda, doubled, 1, beta, projected, 1)
9   .task("bias", Chain::bias, projected, result, 0.25f)
10  .transferToHost(EVERY_EXECUTION, result);
11
12 try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
13   plan.withGridScheduler(grid).withCUDAGraph().execute();
14 }
```



One CUDA graph: `scale` → `tile` → `gemv` → `bias`

●●●●●●●● Only result comes back. `scaled`, `doubled` and `projected` never left the GPU.

The whole TaskGraph API on one slide

All of it is Java on the TaskGraph and the execution result: no cudaMalloc, no cudaMemcpy.

What runs

<code>.task(...)</code>	@Parallel loop: TornadoVM picks the thread mapping
<code>.task(..., ctx, ...)</code>	KernelContext first: globalIdx, local memory, barriers
<code>.task(..., tc, ...)</code>	TileContext first: CUDA Tile, the grid counts tile blocks (develop)
<code>.libraryTask(...)</code>	cuBLAS, cuFFT, cuDNN, CUTLASS on the same device buffers

What moves, and when

Call	Copies	Use for
<code>.transferToDevice(FIRST_EXECUTION, a)</code>	on the first run	weights, constants
<code>.transferToDevice(EVERY_EXECUTION, a)</code>	before every run	inputs that change
<code>.transferToHost(EVERY_EXECUTION, a)</code>	after every run	results read every time
<code>.transferToHost(UNDER_DEMAND, a)</code> <code>result.transferToHost(a)</code>	when you call it	iterative state
<code>result.transferToHost(new DataRange(a).withOffset(o).withSize(s))</code>	only the slice	large outputs

Tasks decide what runs. Transfer modes decide what moves and when. Both live in the same few lines of host code.

TornadoVM SDK

Get started in <1 minute!

Quick install TornadoVM 7.1.0

Three ways to get started, with CUDA as the first-class NVIDIA backend.

1 Through SDKMAN!

Recommended

```
$ sdk list tornadovm
$ sdk install tornadovm 7.1.0-jdk22plus-cuda
```

2 Download the SDK directly

```
https://www.tornadovm.org/downloads
```

3 Clone & build from source

```
$ git clone https://github.com/beehive-lab/TornadoVM.git
$ cd TornadoVM && make BACKEND=cuda
$ source setvars.sh
```

Available versions

7.1.0-jdk22plus-cuda

latest

7.1.0-jdk21-cuda

7.1.0-jdk22plus-openc1

7.1.0-jdk22plus-metal

Download TornadoVM SDK

7.1.0 · JDK 22+

CUDA

linux/x86_64

Download

```
// module-info.java
module my.app { requires tornado.api;
}
```

Demo · SDKMAN!

On the MacBook, with the Metal SDK.

● ● ● Terminal

```
$ sdk list tornadovm
$ sdk install tornadovm 7.1.0-jdk22plus-metal
$ tornado --devices
$ tornado --version

$ tornado -cp $TORNADOVM_HOME/share/java/tornado/tornado-examples-7.1.0.jar
uk.ac.manchester.tornado.examples.compute.MatrixVectorRowMajor

# the bridge for IDEs, Maven, Gradle and nsys
$ tornado --generate-argfile
$ cat $TORNADOVM_HOME/tornado-argfile
```

What to point at

- 01 The version grid: 7.1.0 × jdk21 / jdk22plus × opencl, cuda, metal
- 02 --devices: 0:0 is backend:device, used later with -Dtornado.device

Hands-on-Lab Repository



Scan for Lab 01

Demo · Debug with TornadoInsight

Catch unsupported Java as you type, and run one task from the IDE.

```
void broken(FloatArray in, FloatArray out, int n) {  
    for (@Parallel int i = 0; i < n; i++) {  
        String s = "not on a GPU";          // a data type  
        if (in.get(i) < 0)  
            throw new RuntimeException();    // an exception  
        out.set(i, recurse(in.get(i)));    // recursion  
    }  
}  
  
static float recurse(float x) {  
    if (x < 1.0f) {  
        return x;  
    }  
    return recurse(x / 2.0f) + 1.0f;  
}
```

The plugin finds all three faults.

In IntelliJ

- 01 Install from JetBrains Marketplace:
<https://plugins.jetbrains.com/plugin/23309-tornadoinsight>
- 02 Settings → TornadoInsight: Configure an array size for test inputs
- 03 The static checker flags the String, the throw and the recursion as you type
- 04 Select the method in the TornadoInsight window to run it: no main, no TaskGraph, and the generated kernel is printed

Hands-on-Lab Repository



Scan for Lab 03

The profiler: where the time actually goes

One flag, every phase of a task graph, from compilation to copy-out.

● ● ● Every timer is in nanoseconds

```
$ tornado --enableProfiler console -m tornado.examples/...VectorAddInt 100000
```

```
{ "s0": {  
  "TOTAL_TASK_GRAPH_TIME":      "285317518",  
  "TOTAL_GRAAL_COMPILE_TIME":   "109520628",  
  "TOTAL_DRIVER_COMPILE_TIME":  "63298322",  
  "TOTAL_CODE_GENERATION_TIME": "16564279",  
  "COPY_IN_TIME":               "72044",  
  "COPY_OUT_TIME":              "24153",  
  "TOTAL_KERNEL_TIME":          "47974"  
}}
```

285 ms

total task-graph time first execution

190 ms

compilation Graal + driver + code generation

0.05 ms

kernel time the part you were optimising

Run it in a loop and the compile timers disappear from the second run: the binary is cached. Java developers already know this shape from the JIT.

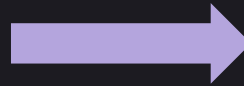
Switching to the Linux box

Same Java, a different SDK. Everything from here needs the CUDA backend.

MacBook

Metal SDK

SDK packaging
Metal and TornadoInsight
the API animations



Linux box

CUDA SDK · NVIDIA GPU

CUDA and TileContext
the Hybrid API and cuBLAS
jitLLM, DevovxGenie, Flink



From here on: CUDA 13.3+, driver R580+, compute capability 8.0+ for CUDA Tile. Everything else needs any recent NVIDIA GPU.

06

The Hybrid API

your kernels + NVIDIA's

Java kernels, CUDA Tile kernels and vendor-optimised libraries in one task graph, on shared GPU buffers.

Demo · CUDA backend, TileContext and cuBLAS

On the Linux box, with the 7.1.0 CUDA SDK, and **NVIDIA Profiling Tools** (nsys).

Hands-on-Lab Repository



Scan for Lab 04

8 NVIDIA libraries, one call shape

Each provider is a Java module, discovered by ServiceLoader and bound over Panama FFM.

cuBLAS

Sgemv · Sgemm · TF32 · strided-batched · FP16 / BF16 GemmEx

cuBLASLt

FP32 / FP16 / FP8 matmul · fused bias + GELU epilogues

cuFFT

C2C · R2C · C2R · batched and 2D transforms

cuVS

Vector store, vector indexing

cuDNN

conv2d · softmax · ReLU / sigmoid / tanh · max-pool · SDPA

cuSPARSE

CSR SpMV and SpMM

CUTLASS

S / H / B-GEMM · fused bias + ReLU, GELU, SiLU and more

cuDF

Data frames, RAPIDS

Same shape for cuFFT: forward FFT → Java filter → inverse FFT, and the spectrum never leaves the GPU.

Six NVIDIA libraries, one call shape

Each provider is a Java module

cuBLAS

Sgemv · Sgemm · TF32 · strided-batched · FP16 / BF16 GemmEx

cuDNN

conv2d · softmax · ReLU / sigmoid / tanh · max-pool · SDPA

Same shape for cuFFT: forward FFT

 Agent Engineering & Tooling

 Tools-in-Action 30min

 INTERMEDIATE




Tap into the NVIDIA Ecosystem from Java with TornadoVM's Hybrid API

TornadoVM's Hybrid API lets Java developers combine GPU kernels written in Java with direct access to NVIDIA libraries like CUDA, cuBLAS, and cuFFT—without JNI. The session shows how to build high-performance AI/HPC apps in Java while preserving portability, productivity, and incremental acceleration within the JVM.

AI-generated summary — may contain inaccuracies

 **Thanos Stratikopoulos**
Beestream.ai | TornadoVM

 **Michalis Papadimitriou**
Neo4j

 **Mary Xekalaki**
beestream.ai | TornadoVM

WHEN & WHERE



Tuesday, October 6, 16:50-17:20 ⓘ

📍 Room 4

ROOM OCCUPANCY

NO OCCUPANCY INFO YET

Integration with real-world frameworks (AI, Data Analytics)

Advanced Session

Java Inference Tornado toolkit

1st Java-native LLM inference toolkit that automatically compiles and executes on GPUs via TornadoVM

j ⚡ tLLM

 @jitllm

 @jitllm.io

 <https://github.com/beehive-lab/jitllm>

Java Inference Tornado tool

1st Java-native LLM inference toolkit that automatically compiles

j ⚡ tLLM



@jitllm



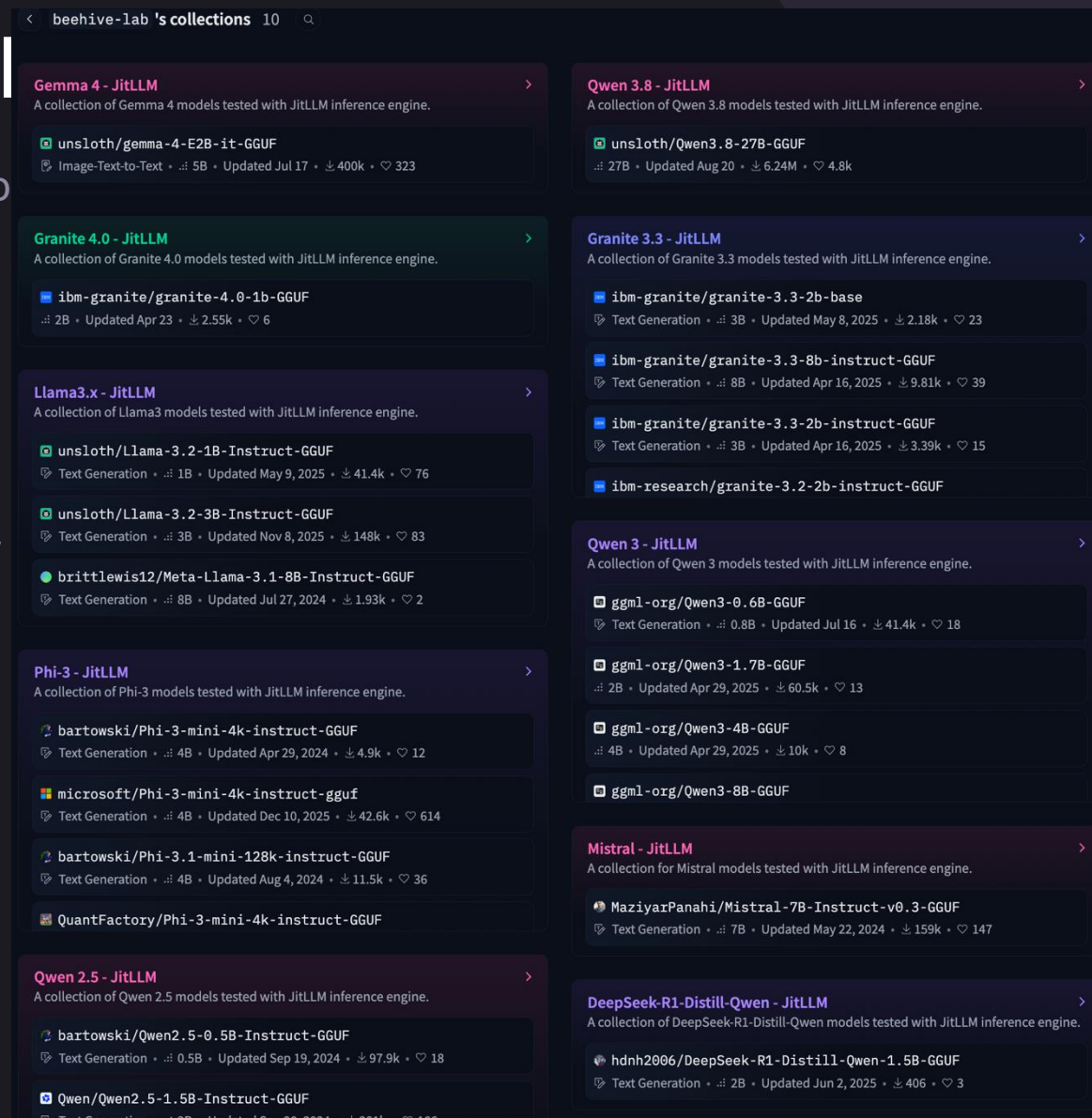
@jitllm.io



<https://github.com/beehive-lab/jitllm>



<https://huggingface.co/beehive-lab/collections>



The screenshot displays the 'beehive-lab' collections page on Hugging Face, featuring a grid of LLM collections. Each collection entry includes the model name, a brief description, and a list of GGUF files with their respective sizes, update dates, and download counts. An arrow points from the text 'https://huggingface.co/beehive-lab/collections' to this screenshot.

Collection Name	Description	Files
Gemma 4 - JitLLM	A collection of Gemma 4 models tested with JitLLM inference engine.	unsloth/gemma-4-E2B-it-GGUF: Image-Text-to-Text • 5B • Updated Jul 17 • 400k • 323
Qwen 3.8 - JitLLM	A collection of Qwen 3.8 models tested with JitLLM inference engine.	unsloth/Qwen3.8-27B-GGUF: 27B • Updated Aug 20 • 6.24M • 4.8k
Granite 4.0 - JitLLM	A collection of Granite 4.0 models tested with JitLLM inference engine.	ibm-granite/granite-4.0-1b-GGUF: 2B • Updated Apr 23 • 2.55k • 6
Granite 3.3 - JitLLM	A collection of Granite 3.3 models tested with JitLLM inference engine.	- ibm-granite/granite-3.3-2b-base: Text Generation • 3B • Updated May 8, 2025 • 2.18k • 23 - ibm-granite/granite-3.3-8b-instruct-GGUF: Text Generation • 8B • Updated Apr 16, 2025 • 9.81k • 39 - ibm-granite/granite-3.3-2b-instruct-GGUF: Text Generation • 3B • Updated Apr 16, 2025 • 3.39k • 15 - ibm-research/granite-3.2-2b-instruct-GGUF
Llama3.x - JitLLM	A collection of Llama3 models tested with JitLLM inference engine.	- unsloth/Llama-3.2-1B-Instruct-GGUF: Text Generation • 1B • Updated May 9, 2025 • 41.4k • 76 - unsloth/Llama-3.2-3B-Instruct-GGUF: Text Generation • 3B • Updated Nov 8, 2025 • 148k • 83 - brittlewis12/Meta-Llama-3.1-8B-Instruct-GGUF: Text Generation • 8B • Updated Jul 27, 2024 • 1.93k • 2
Phi-3 - JitLLM	A collection of Phi-3 models tested with JitLLM inference engine.	- bartowski/Phi-3-mini-4k-instruct-GGUF: Text Generation • 4B • Updated Apr 29, 2024 • 4.9k • 12 - microsoft/Phi-3-mini-4k-instruct-gguf: Text Generation • 4B • Updated Dec 10, 2025 • 42.6k • 614 - bartowski/Phi-3.1-mini-128k-instruct-GGUF: Text Generation • 4B • Updated Aug 4, 2024 • 11.5k • 36 - QuantFactory/Phi-3-mini-4k-instruct-GGUF
Qwen 2.5 - JitLLM	A collection of Qwen 2.5 models tested with JitLLM inference engine.	- bartowski/Qwen2.5-0.5B-Instruct-GGUF: Text Generation • 0.5B • Updated Sep 19, 2024 • 97.9k • 18 - Qwen/Qwen2.5-1.5B-Instruct-GGUF: Text Generation • 1.5B • Updated Sep 30, 2024 • 281k • 155
Qwen 3 - JitLLM	A collection of Qwen 3 models tested with JitLLM inference engine.	- ggml-org/Qwen3-0.6B-GGUF: Text Generation • 0.6B • Updated Jul 16 • 41.4k • 18 - ggml-org/Qwen3-1.7B-GGUF: 2B • Updated Apr 29, 2025 • 60.5k • 13 - ggml-org/Qwen3-4B-GGUF: 4B • Updated Apr 29, 2025 • 10k • 8 - ggml-org/Qwen3-8B-GGUF
Mistral - JitLLM	A collection for Mistral models tested with JitLLM inference engine.	MaziyarPanahi/Mistral-7B-Instruct-v0.3-GGUF: Text Generation • 7B • Updated May 22, 2024 • 159k • 147
DeepSeek-R1-Distill-Qwen - JitLLM	A collection of DeepSeek-R1-Distill-Qwen models tested with JitLLM inference engine.	hdnh2006/DeepSeek-R1-Distill-Qwen-1.5B-GGUF: Text Generation • 2B • Updated Jun 2, 2025 • 406 • 3

Demo · jitLLM on your own GPU

jitLLM, formerly GPULlama3.java. Metal, OpenCL or CUDA is picked from your SDK.

● ● ● Terminal

```
$ M=gemma-4-E2B-it-Q8_0.gguf

# no build: a released TornadoVM SDK plus JBang
$ jbang jitllm@beehive-lab -m $M \
    -p "Explain GPU acceleration in one sentence."

# from a clone
$ git clone https://github.com/beehive-lab/jitllm.git && cd jitllm && make
$ ./jitllm --gpu --verbose --model $M \
    --prompt "Explain the benefits of GPU acceleration."
$ ./jitllm serve -m $M --gpu --port 8090
```

Hands-on-Lab Repository



Scan for Lab 06



jitLLM, then analyze the execution chain

Same flags, same tools, on a real workload.

Where did the first token go?

Graal compiles every kernel once, on first use, and then never again.

What is the per-token cost?

Kernel time plus dispatch. That is what 6.0.0 and 6.1.0 targeted: #1027 benchmarks
LLM decode by name.

Which kernels are they?

RMS norm, RoPE, attention and the quantised projections: the same kernels the
TileContext examples rewrite.

Terminal

```
$ M=beehive-llama-3.2-1b-instruct-fp16.gguf
# the same TornadoVM flags, through jitLLM
$ ./jitllm --gpu --model $M --prompt "..." \
  --print-kernel
$ ./jitllm --gpu --model $M --prompt "..." \
  --print-taskgraph-chain
```

Profiler options are grouped under "Debug and Profiling" in `./jitllm --help`.

Demo · TornadoVMPulse

A dashboard for the profiler's output.

● ● ● Terminal

```
$ tornado --enableProfiler silent \  
  --dumpProfiler profile.json \  
  -m tornado.examples/uk.ac.manchester.tornado.examples.VectorAddInt 1000000  
  
$ git clone https://github.com/beehive-lab/TornadoVMPulse.git  
$ cd TornadoVMPulse  
$ pip install -r requirements.txt  
$ streamlit run src/app.py          # then upload profile.json
```

What to point at

- 01 The sunburst: kernel time is a sliver of the total on a first run
- 02 Copy-in and copy-out: time and count per task graph
- 03 Power comes from NVML on NVIDIA; on the Mac that panel stays empty

Hands-on-Lab Repository



Scan for Lab 07

Demo · TornadoViz

See the task graph, its transfers and every object's lifetime.

● ● ● Terminal

```
$ tornado --dumpBC ./bytecodes -m tornado.examples/\
    uk.ac.manchester.tornado.examples.VectorAddInt 1000000

$ git clone https://github.com/beehive-lab/TornadoViz
$ cd TornadoViz
$ pip install -r requirements.txt
$ streamlit run tornado-visualizer-fixed.py # load the dump
```

What to point at

- 01 Task-graph dependencies and the data flowing between graphs
- 02 The memory timeline: allocations, host-to-device, device-to-host, deallocations
- 03 Run your own kernel with `FIRST_EXECUTION`, then `EVERY_EXECUTION`: the redundant transfers stand out

Hands-on-Lab Repository



Scan for Lab 08

WHERE WE ARE TODAY

Pure Java, at llama.cpp speed on GPUs

Same GPU, same GGUF weights. Every kernel written in Java and compiled by TornadoVM at runtime.

0.94×

DECODE · 27B

of llama.cpp decode on Qwen3.8-27B

40.5 vs 43.2 tok/s · Q4_0 · pure JIT, zero native code

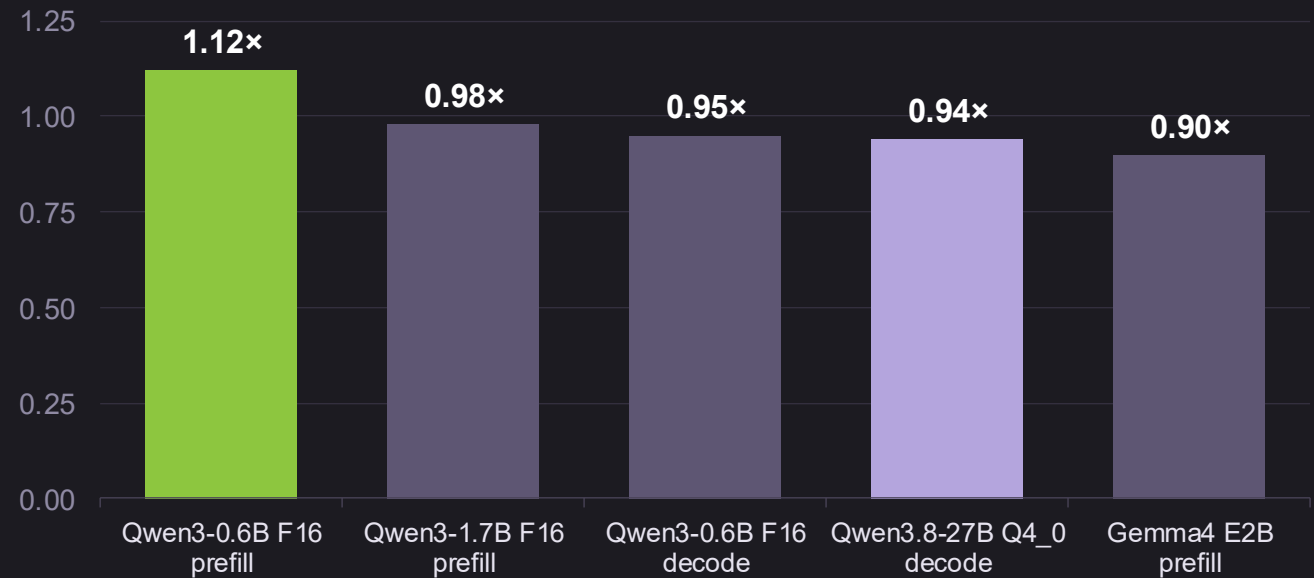
1.12×

PREFILL · HYBRID

faster than llama.cpp on prefill

Qwen3-0.6B F16 pp512 · 50,300 vs 44,805 tok/s · cuBLAS + Java

Throughput relative to llama.cpp = 1.00



5 of 16 measured configurations · llama.cpp at the same batch size · GPU: fill in

WHERE WE ARE TODAY

Pure Java, at 1

Same GPU, same GGUF weights. E

0.94×

of llama.cpp decode on Qwen3.8-27

40.5 vs 43.2 tok/s · Q4_0 · pure JIT, zero native code

1.12×

faster than llama.cpp on prefill

Qwen3-0.6B F16 pp512 · 50,300 vs 44,805 tok/s · cu

 Java Language & Platform

 Conference 50min

 INTERMEDIATE



Building LLM Inference Engines in Java with TornadoVM and the NVIDIA Ecosystem

The session shows how Java can run GPU-accelerated local LLMs natively with TornadoVM, using NVIDIA libraries without JNI. It covers GPULlama3.java, quantized inference, Quarkus and LangChain4J integration, and how TornadoVM plus Project Babylon/Panama position Java as a strong AI and HPC platform.

 AI-generated summary — may contain inaccuracies



Thanos Stratikopoulos
Beestream.ai | TornadoVM



Michalis Papadimitriou
Neo4j



Orion Papadakis
TornadoVM

WHEN & WHERE



Wednesday, October 7, 16:40-17:30 ⓘ

 Room 10

ROOM OCCUPANCY
NO OCCUPANCY INFO YET

jitLLM in the Java AI and data stack

One local GPU engine, plugged into the frameworks and databases Java teams already use.

AI FRAMEWORK

LangChain4j

Official model provider since v1.7.1

Available



AI FRAMEWORK

Quarkus LangChain4j

jitLLM as a Quarkus inference engine

Available



AI FRAMEWORK

Spring AI

Spring apps on the local GPU, through jitLLM

In progress



IDE PLUGIN

DevovxGenie

IntelliJ assistant over jitllm serve

In progress



DATA ANALYTICS

Apache Flink

SQL offload plus a model in the same JVM

In progress

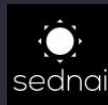


DATABASE

PostgreSQL

Local GPU inference for PostgreSQL data

Available



FINANCIAL SERVICES

Any FinTech platform

Trading systems, risk engines

In progress

AND MORE

Any OpenAI client

jitllm serve speaks /v1/chat/completions

Available



DevovxGenie: a local LLM inside your IDE

IntelliJ talks to jitLLM over its OpenAI-compatible endpoint, on your own GPU.

Wire it up

- 01 Start the engine: `./jitllm serve -m <model>.gguf --gpu --port 8090`
- 02 IntelliJ: install the DevovxGenie fork build from disk (repo below)
- 03 Provider: Custom OpenAI · Base URL: `http://localhost:8090/v1` · any non-empty API key
- 04 Select a method, ask for a test, and watch jitLLM stream the answer

The plugin is Java. The engine is Java. The kernels were compiled from Java.

Nothing leaves the machine, and the only non-Java piece left is the vendor driver.

IntelliJ + DevovxGenie Java, built on LangChain4j

HTTP localhost:8090/v1, OpenAI-compatible

jitllm serve Java, /v1/chat/completions

TornadoVM kernels compiled from Java

GPU driver reached through java.lang.foreign

Hands-on-Lab Repository



Scan for Lab 10

Integration with Apache Flink Table API

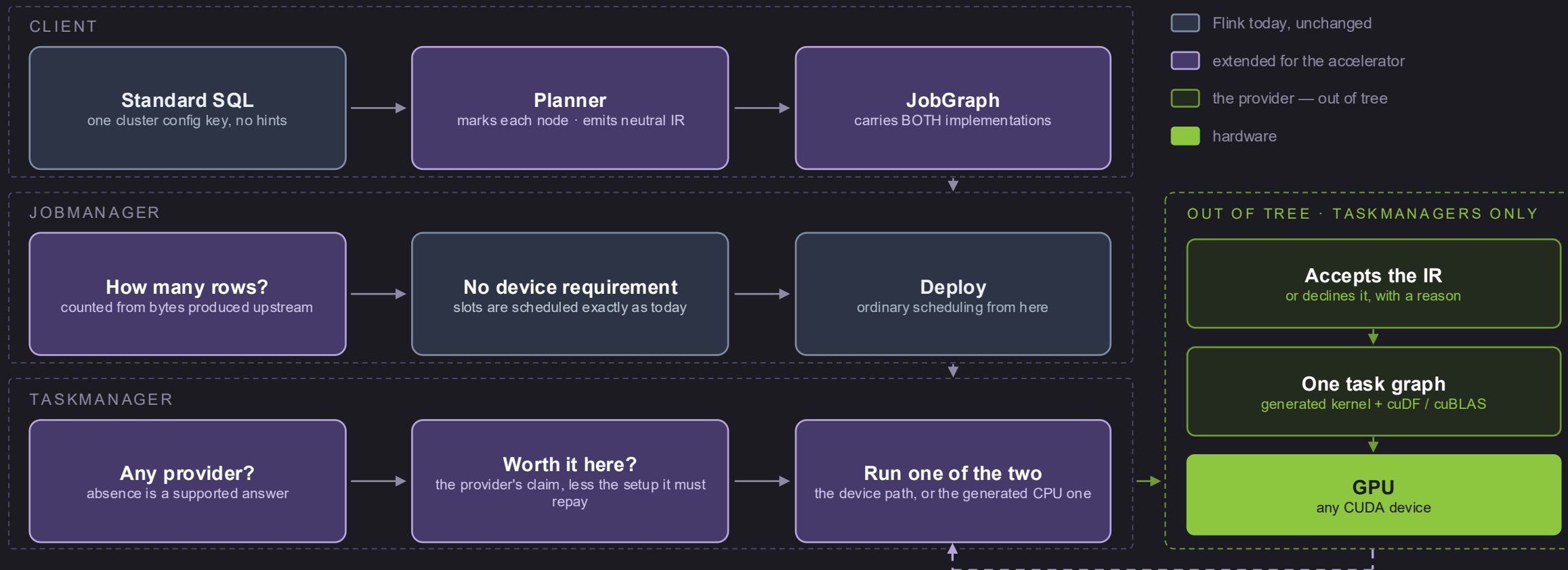
Enabling seamless GPU acceleration via TornadoVM





A RAPIDS-like path for Apache Flink

Read down: a query is planned, then scheduled, and only then meets a device.



- Flink today, unchanged
- extended for the accelerator
- the provider — out of tree
- hardware



Demo · An SQL operator on the GPU

Ordinary Flink SQL, one config key, and the CUDA it compiled to.

● ● ● Terminal

```
$ ./demo-haversine.sh --print-kernel
# in IntelliJ, alongside:
#   HaversineSQLExample · the SQL and the three settings
#   Apache Flink log files
```

What to point at

- 01 The query is ordinary SQL. The only thing written for the GPU is NOT NULL in the DDL.
- 02 AccelKernelGenerator turns the expression tree into a @Parallel Java method
- 03 --print-kernel prints CUDA that exists nowhere in the repository

Hands-on-Lab Repository

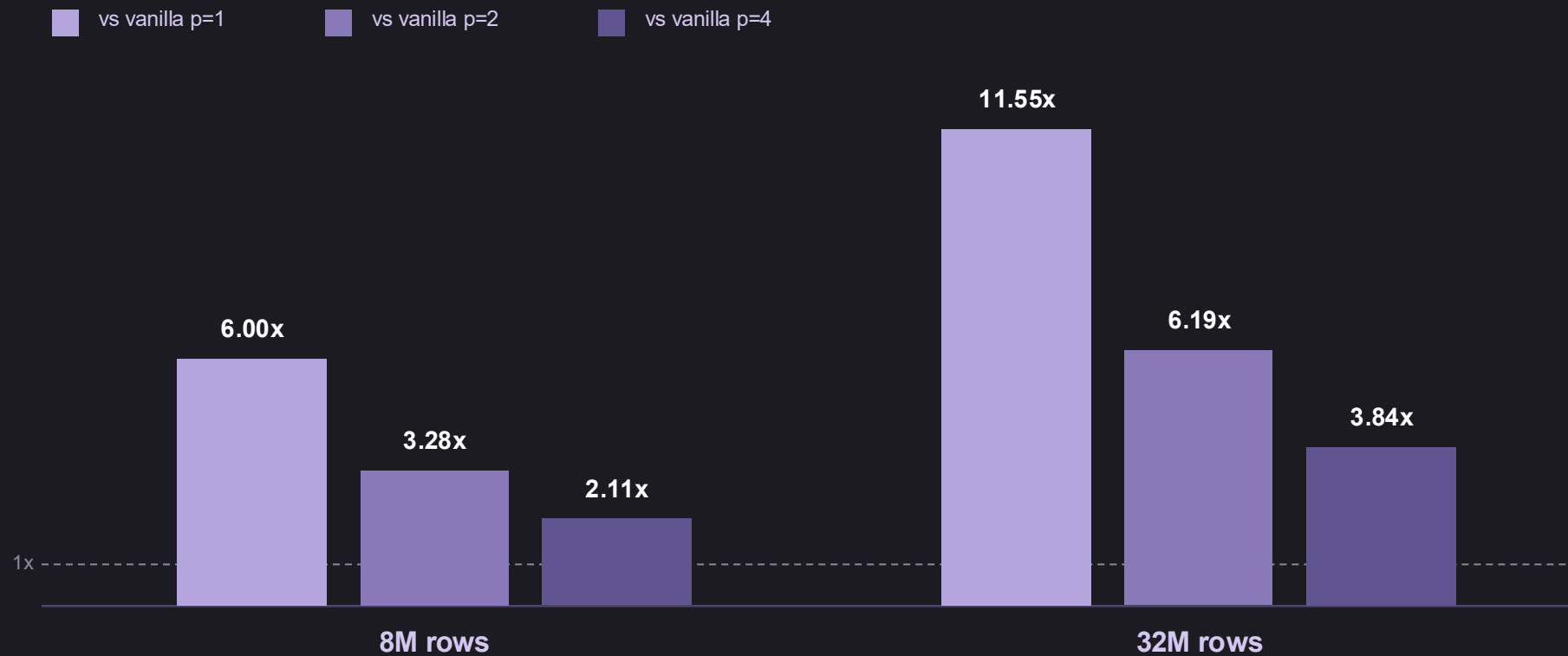


Scan for Lab 11



Haversine: a lead four CPU slots do not close

End-to-end wall clock of `./bin/flink run` · one GPU at parallelism 1 · vanilla Flink at 1, 2 and 4 slots.



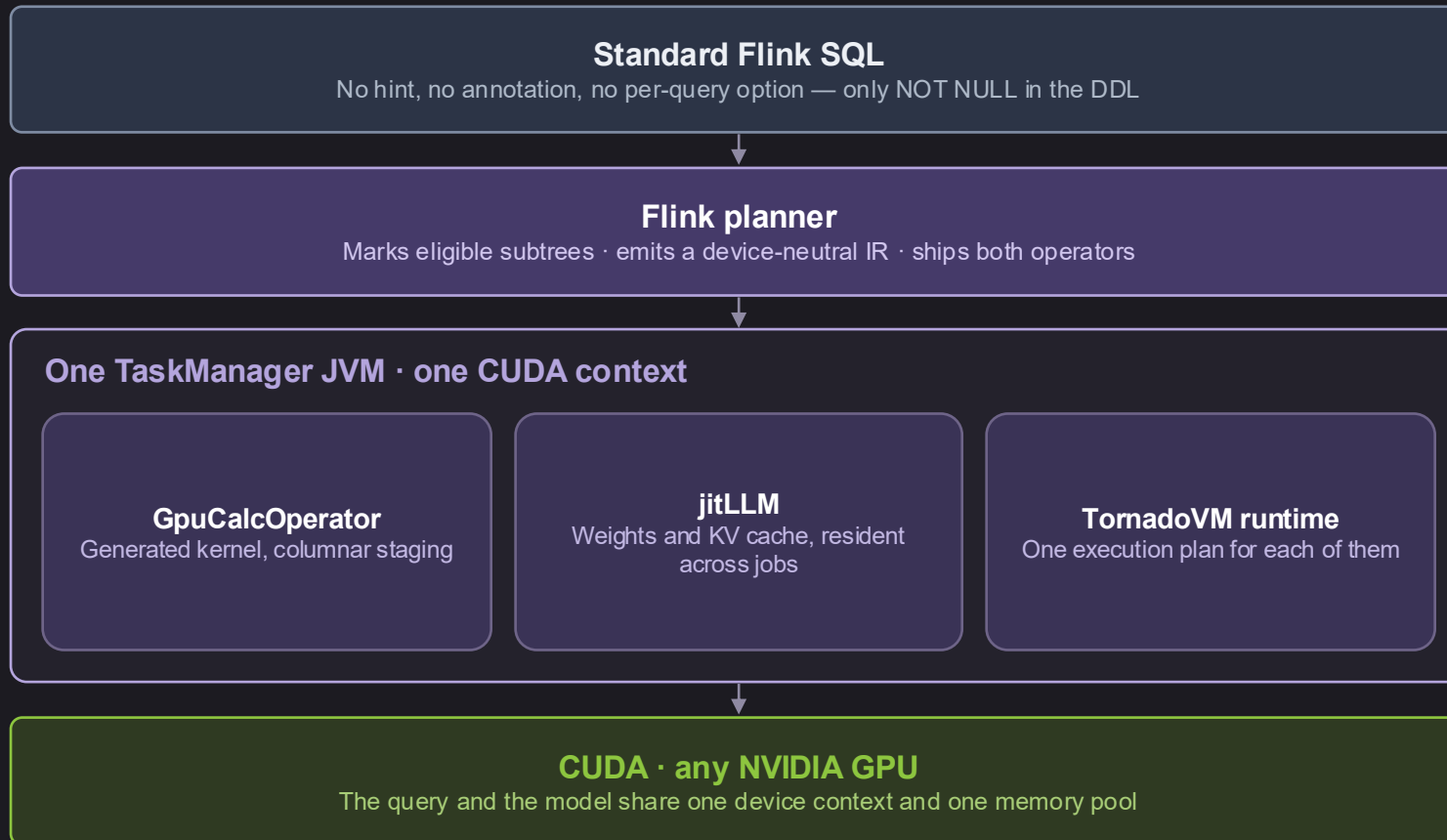
Device 11.5 s at 8M and 22.3 s at 32M; vanilla 69 s → 257 s. The device arm is mostly fixed cost, so the lead grows with the data.

RTX 4070 Laptop · CUDA 12.5 · JDK 21 · Flink 2.3.0 · device median of 3 runs, vanilla mean of 2 · both arms returned the full row count.



Query and model in the same JVM

Three things in the same JVM, sharing one device context.



Why staying in the JVM matters

- ✓ In the usual arrangement the model lives in another process, so the data has to be broken up to reach it: rows become text, the text becomes an HTTP request, and the engine on the other side has its own context and its own copy. That holds for any external inference server.
- ✓ Here nothing leaves the device between the two halves. The kernel scores eight million readings and the model reads the digest that comes out of it — same GPU, same data, no socket, no serialisation, no second copy.
- ✓ One CUDA context and one memory pool, with the weights resident across jobs. Preprocessing and inference stop being two systems that have to be joined.



Demo · Fleet telemetry triage

Eight million readings scored on the device, then a model writes the note.

● ● ● Terminal

```
$ ./demo-llm.sh
# starts a cluster, runs the job once, stops it
# in IntelliJ, alongside:
#   TelemetryTriage.java
#   GpuTriagePipeline.java
```

What to point at

- 01 The model is a scalar UDF, and the scoring is the same offload path as the first demo. Nothing is special-cased for LLMs.
- 02 The hand-off: the GROUP BY output goes straight into jitLLM as a Java object, with no process boundary in between.
- 03 A SQL engine, a GPU compiler and a language model composed in one JVM on one card — driven by ordinary Flink SQL.

Hands-on-Lab Repository



Scan for Lab 12



Fleet telemetry: where the time goes

Against Flink + CPU preprocessing + llama.cpp · Qwen3-0.6B FP16 · warm.

9×

faster preprocessing

0.92 s device, 8.44 s cores

2.06×

end to end, warm

6.84 s against 14.08 s

−0.38 s

given back on inference

121 tok/s vs llama.cpp's 148

TornadoVM for Kotlin

WORK IN PROGRESS · PR #1123

Kotlin is JVM bytecode too, so Kotlin kernels run on OpenCL, CUDA and Metal.

● ● ● VectorAdd.kt

```
fun vectorAdd(a: TFloatArray, b: TFloatArray, c: TFloatArray) {
    parallelFor(0, c.size) { i -> c[i] = a[i] + b[i] }
}

val graph = taskGraph("s0") {
    transferToDevice(FIRST_EXECUTION, a, b)
    task("t0", ::vectorAdd, a, b, c)
    transferToHost(EVERY_EXECUTION, c)
}
graph.toExecutionPlan().use { it.execute() }
// parallelFor stands in for @Parallel, which kotlinc drops
```

What the PR adds

- 01 **Opt-in build** make KOTLIN=1. The default build never downloads the Kotlin toolchain.
- 02 **Guarded runtime** rewrites touch only kotlinc classes; off with -Dtornado.kotlin.support=false
- 03 **Beyond loops** KernelContext, @Reduce, cuBLAS and CUDA Tile ($a + b \rightarrow tc.add(a, b)$)
- 04 **Same kernels as Java** Kotlin/Java run time 0.99–1.12× on seven ported benchmarks

Hands-on-Lab Repository



Scan for Lab 13

Takeaways

What to take back to your team.

- 01 Kernels in Java:** threads or tiles, compiled to inspectable CUDA within a few percent of hand-written code
- 02 NVIDIA libraries as graph nodes:** cuBLAS, cuFFT, cuDNN, cuSPARSE, CUTLASS, cuDF. no JNI, no bindings to maintain
- 03 One CUDA graph for everything:** JIT, CUDA Tile and library nodes captured and replayed together
- 04 jitLLM closes the gap with llama.cpp:** Hybrid API and CUDA features from TornadoVM boost performance for LLMs
- 05 Incremental, on your JDK:** one hot loop at a time, on any JDK 21–27
- 06 Integrated with other frameworks:** LangChain4j, Quarkus-LangChain4j, Apache Flink, Spring AI, DevoxGenie...

Thank you! Questions?

Resources to take home.

1

TornadoVM

github.com/beehive-lab/TornadoVM

2

jitLLM

github.com/beehive-lab/jitllm

3

Demos from this talk

<https://github.com/beehive-lab/tornadovm-hands-on-lab>

4

Docker image

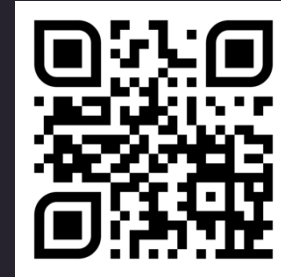
<https://hub.docker.com/r/beehivelab/tornadovm-nvidia-openjdk>

5

SDKMAN!

sdkman.io/sdks/tornadovm

Scan to get started



beestream.ai

GET IN TOUCH

 LinkedIn

[/company/beestream-ai](https://www.linkedin.com/company/beestream-ai)

 X

[@beestream_ai](https://twitter.com/beestream_ai)

 Bluesky

[@beestream.ai](https://bsky.app/profile/beestream.ai)

 Open source

[beehive-lab/TornadoVM](https://github.com/beehive-lab/TornadoVM)

We value your feedback!

Rate this talk



DEVOXX™